sshmount ソースリスト

美都

2026 年 10 月 13 日

目次

1 メインモジュール　main.rs ・・・

1　メインモジュール　main.rs

```rust
mod cmdline_opt;
mod fuse_util;
mod ssh_connect;
mod ssh_filesystem;

use anyhow::{Context, Result};
use clap::Parser;
use cmdline_opt::Opt;
use daemonize::Daemonize;
use fuse_util::{make_full_path, make_mount_option, make_remote_path};
use log::{error, info};
use ssh_connect::make_ssh_session;

fn main() -> Result<()> {
    //env_logger::init();
    let opt = Opt::parse();

    systemd_journal_logger::JournalLog::new()
        .expect("Fail initialize logger")
        .with_syslog_identifier("sshmount".to_string())
        .install()
        .expect("Fail install logger");
    match opt.debug {
        true => log::set_max_level(log::LevelFilter::Debug),
        false => log::set_max_level(log::LevelFilter::Info),
    }

    let ssh = make_ssh_session(&opt).context("Failed to generate ssh session.")?;

    let path = make_remote_path(&opt, &ssh).context("Failed to generate remote path.")?;
    let options = make_mount_option(&opt);
    let mut config = fuser::Config::default();
    config.mount_options = options;

    let mount_point = make_full_path(&opt.mount_point)?;

    // プロセスのデーモン化
    if opt.daemon {
        let daemonize = Daemonize::new();
        if let Err(e) = daemonize.start() {
            error!("daemonization filed.(error: {})", e);
            eprintln!("daemonization filed.(error: {})", e);
        }
```

```
    }
    // ファイルシステムへのマウント実行
    let fs = ssh_filesystem::Sshfs::new(ssh, &path).map_err(|e| {
        error!("Failed to create Sshfs.(error: {})", e);
        anyhow::anyhow!("Failed to create Sshfs.(error: {})", e)
    })?;
    //fuser::mount(fs, mount_point, &config).context("Failed to mount FUSE.")?;
    let handle = fuser::spawn_mount(fs, mount_point, &config).map_err(|e| {
        error!("Failed to mount FUSE.(error: {})", e);
        anyhow::anyhow!("Failed to mount FUSE.(error: {})", e)
    })?;
    info!("FUSE mounted successfully.");
    match handle.join() {
        Ok(()) => {
            info!("FUSE unmounted successfully.");
            Ok(())
        }
        Err(e) => {
            error!("Failed to unmount FUSE.(error: {:?})", e);
            eprintln!("Failed to unmount FUSE.(error: {:?})", e);
            Err(anyhow::anyhow!("Failed to unmount FUSE.(error: {:?})", e))
        }
    }
}
```

2　コマンドラインオプションの解析関係

2.1　コマンドラインオプションの定義 cmdline_opt.rs

```rust
pub mod remote_host;

use anyhow::{Context, anyhow};
use clap::Parser;
use std::path::PathBuf;

use remote_host::RemoteName;

/// コマンドラインオプション
#[derive(Parser, Debug)]
#[command(author, version, about)]
pub struct Opt {
    /// Distination [user@]host:[path] or scp://[user@]host[:port][/path]
    pub remote: RemoteName,
    /// Path to mount
    #[arg(value_parser = exist_dir)]
    pub mount_point: String,
    /// Path to config file
    #[arg(short = 'F', long)]
    pub config_file: Option<PathBuf>,
    /// Login name
    #[arg(short, long)]
    pub login_name: Option<String>,
    /// File name of secret key file
    #[arg(short, long)]
    pub identity: Option<PathBuf>,
    /// Port no
    #[arg(short, long)]
    pub port: Option<u16>,
    /// Read only
    #[arg(short, long)]
    pub readonly: bool,
    /// Not executable
    #[arg(long)]
    pub no_exec: bool,
    /// Do not change access date and time(atime)
    #[arg(long)]
    pub no_atime: bool,
    /// run in daemon mode
    #[arg(short, long)]
    pub daemon: bool,
```

```rust
    /// Set the log level to debug mode
    #[arg(long)]
    pub debug: bool,
}

/// 指定されたディレクトリが存在し、中にファイルがないことを確認する。
fn exist_dir(s: &str) -> anyhow::Result<String> {
    match std::fs::read_dir(s) {
        Ok(mut dir) => match dir.next() {
            None => Ok(s.to_string()),
            Some(_) => Err(anyhow!("Mount destination directory is not empty.")),
        },
        Err(e) => match e.kind() {
            std::io::ErrorKind::NotFound => Err(anyhow!("The mount directory does not exist.")),
            std::io::ErrorKind::NotConnected => Err(anyhow!(
                "The network of the mount directory is disconnected. (Did you forget to umount?)."
            )),
            _ => Err(e).context("Unexpected error.(check mount directory)"),
        },
    }
}

#[cfg(test)]
mod test {
    use super::*;
    #[test]
    fn verify_cli() {
        use clap::CommandFactory;
        Opt::command().debug_assert()
    }
}
```

2.2 コマンドライン　remotehost の解析部 cmdline_opt/remote_host.rs

```rust
//! コマンドラインオプションにおける RemoteHost 構造体

use fluent_uri::{Uri, component::Host};
use log::debug;
use std::net::IpAddr;

/// コマンドラインの接続先ホスト情報
#[derive(Clone, Debug, PartialEq)]
pub struct RemoteName {
    /// ユーザー名
    pub user: Option<String>,
    /// IP アドレス
    pub host: HostInfo,
    /// ポート番号
    pub port: Option<u16>,
    /// 接続先パス
    pub path: Option<std::path::PathBuf>,
}

/// ホスト情報 (ホスト名または、IP アドレス)
#[derive(Clone, Debug, PartialEq)]
pub enum HostInfo {
    Name(String),
    Ip(IpAddr),
}

impl std::fmt::Display for HostInfo {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            HostInfo::Name(name) => write!(f, "{}", name),
            HostInfo::Ip(ip) => write!(f, "{}", ip),
        }
    }
}

impl RemoteName {
    /// remote 引数の解析 (URI 形式の場合)
    fn parse_uri(s: &str) -> Result<Self, ErrorRemoteName> {
        // URI の解析
        let uri = Uri::parse(s)?;
        let info = uri
            .authority()
            .ok_or(ErrorRemoteName::NotFoundHostInformationInURI)?;
        // ポート番号の取得
```

```
        let port = info
            .port_to_u16()
            .map_err(|_| ErrorRemoteName::InvalidPortNo)?;
        // ホストアドレスの取得
        let host = match info.host_parsed() {
            Host::Ipv4(ip) => HostInfo::Ip(ip.into()),
            Host::Ipv6(ip) => HostInfo::Ip(ip.into()),
            _ => HostInfo::Name(info.host().to_string()),
        };
        // パス名の取得
        let path_str = uri.path().as_str();
        let path = if path_str.is_empty() {
            None
        } else {
            Some(
                path_str
                    .trim()
                    .parse::<std::path::PathBuf>()
                    .map_err(|_| ErrorRemoteName::InvalidPath)?,
            )
        };
        // ユーザー名の取得
        let user = info.userinfo().and_then(|s| {
            let trimed = s.as_str().trim();
            if trimed.is_empty() {
                None
            } else {
                Some(trimed.to_string())
            }
        });

        Ok(Self {
            user,
            host,
            port,
            path,
        })
    }

    /// remote引数の解析（URI形式でない場合）
    fn parse_non_uri(s: &str) -> Result<Self, ErrorRemoteName> {
        let mut rest_str = s.trim();
        // ユーザー名の取得
        let user = match rest_str.split_once("@") {
            Some((u, r)) => {
                rest_str = r;
```

```
            Some(u.trim().to_string())
        }
        None => None,
    };
    // ホストアドレスの取得
    let host_str: &str;
    // IPv6 の判定
    rest_str = rest_str.trim_start();
    if rest_str.starts_with('[') {
        let (ip6_str, rest) = match rest_str.split_once(']') {
            Some((l, r)) => (l.trim_start_matches('['), r),
            None => return Err(ErrorRemoteName::MissingClosingBracketInIPv6),
        };
        host_str = ip6_str.trim();
        let (_, r) = rest.split_once(":").ok_or(ErrorRemoteName::NoColon)?;
        rest_str = r;
    } else {
        let (h, r) = rest_str.split_once(':').ok_or(ErrorRemoteName::NoColon)?;
        if h.trim().is_empty() {
            return Err(ErrorRemoteName::NoHostName);
        }
        host_str = h.trim();
        rest_str = r;
    }

    let try_ip = host_str.parse::<IpAddr>();
    let host = match try_ip {
        Ok(addr) => HostInfo::Ip(addr),
        Err(_) => HostInfo::Name(host_str.to_string()),
    };

    // パス名の取得
    rest_str = rest_str.trim();
    let path = if rest_str.is_empty() {
        None
    } else {
        Some(
            rest_str
                .parse::<std::path::PathBuf>()
                .map_err(|_| ErrorRemoteName::InvalidPath)?,
        )
    };

    Ok(Self {
        host,
        port: None,
```

```
137                 user,
138                 path,
139             })
140         }
141     }
142
143     impl std::fmt::Display for RemoteName {
144         fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
145             let s = format!(
146                 "<{:?}><{:?}><{:?}><{:?}>",
147                 self.user, self.host, self.port, self.path
148             );
149             s.fmt(f)
150         }
151     }
152
153     impl std::str::FromStr for RemoteName {
154         type Err = ErrorRemoteName;
155         fn from_str(s: &str) -> Result<Self, Self::Err> {
156             let ret = if s.trim_start().starts_with("scp://") {
157                 Self::parse_uri(s)
158             } else {
159                 Self::parse_non_uri(s)
160             };
161             debug!("RemoteName: {:?}", ret);
162             ret
163         }
164     }
165
166     #[derive(thiserror::Error, Debug, PartialEq, Eq)]
167     //#[error("The format of the host to connect to is \"[user@]host:[path]\" or
    ↪  \"scp://[user@]host[:port][/path]\".")]
168     pub enum ErrorRemoteName {
169         #[error("URI parse error.")]
170         UriParse(#[from] fluent_uri::ParseError),
171         #[error("Host information not found at the specified URI.")]
172         NotFoundHostInformationInURI,
173         #[error("Invalid port number.")]
174         InvalidPortNo,
175         #[error("Invalid path name")]
176         InvalidPath,
177         #[error("The closing bracket is missing from IPv6.")]
178         MissingClosingBracketInIPv6,
179         #[error("In the host information, there is no subsequent colon (:).")]
180         NoColon,
181         #[error("No hostname specified.")]
```

```
182     NoHostName,
183 }
184
185 #[cfg(test)]
186 mod test {
187     use fluent_uri::ParseErrorKind;
188
189     use super::*;
190
191     #[test]
192     fn test_from_str_remotename() {
193         use std::path::Path;
194         let s = "mito@reterminal.local:/home/mito";
195         let r: RemoteName = s.parse().unwrap();
196         let k = RemoteName {
197             user: Some("mito".to_string()),
198             host: HostInfo::Name("reterminal.local".to_string()),
199             port: None,
200             path: Some(Path::new("/home/mito").into()),
201         };
202         assert_eq!(r, k);
203
204         let s = "mito@reterminal.local:/home/mito/";
205         let r: RemoteName = s.parse().unwrap();
206         let k = RemoteName {
207             user: Some("mito".to_string()),
208             host: HostInfo::Name("reterminal.local".to_string()),
209             port: None,
210             path: Some(Path::new("/home/mito").into()),
211         };
212         assert_eq!(r, k);
213
214         let s = "mito@[fe80::a00:27ff:fe0e:8c0c]:/home/mito/";
215         let r: RemoteName = s.parse().unwrap();
216         let k = RemoteName {
217             user: Some("mito".to_string()),
218             host: HostInfo::Ip("fe80::a00:27ff:fe0e:8c0c".parse().unwrap()),
219             port: None,
220             path: Some(Path::new("/home/mito").into()),
221         };
222         assert_eq!(r, k);
223
224         let s = "mito@[::1]:/home/mito/";
225         let r: RemoteName = s.parse().unwrap();
226         let k = RemoteName {
227             user: Some("mito".to_string()),
```

```
            host: HostInfo::Ip("::1".parse().unwrap()),
            port: None,
            path: Some(Path::new("/home/mito").into()),
        };
        assert_eq!(r, k);

        let s = "reterminal.local:";
        let r: RemoteName = s.parse().unwrap();
        let k = RemoteName {
            user: None,
            host: HostInfo::Name("reterminal.local".to_string()),
            port: None,
            path: None,
        };
        assert_eq!(r, k);

        let s = " mito @reterminal.local: ";
        let r: RemoteName = s.parse().unwrap();
        let k = RemoteName {
            user: Some("mito".to_string()),
            host: HostInfo::Name("reterminal.local".to_string()),
            port: None,
            path: None,
        };
        assert_eq!(r, k);

        let s = "reterminal.local";
        let r: Result<RemoteName, ErrorRemoteName> = s.parse();
        assert_eq!(r, Err(ErrorRemoteName::NoColon));

        let s = "mito@reterminal.local";
        let r: Result<RemoteName, ErrorRemoteName> = s.parse();
        assert_eq!(r, Err(ErrorRemoteName::NoColon));

        let s = " mito @: ";
        let r: Result<RemoteName, ErrorRemoteName> = s.parse();
        assert_eq!(r, Err(ErrorRemoteName::NoHostName));
    }

    #[test]
    // scp://[user@]host[:port][/path]  の解析テスト
    fn test_from_str_remotename_uri() {
        let s = "scp://name@hostname.hoge:22/test_path";
        let a = RemoteName {
            user: Some("name".to_string()),
            host: HostInfo::Name("hostname.hoge".to_string()),
```

```
            port: Some(22),
            path: Some(std::path::PathBuf::from("/test_path")),
        };
        let r: RemoteName = s.parse().unwrap();
        assert_eq!(r, a);

        let s = "scp://name@192.168.0.1:22/test_path/path";
        let a = RemoteName {
            user: Some("name".to_string()),
            host: HostInfo::Ip("192.168.0.1".parse().unwrap()),
            port: Some(22),
            path: Some(std::path::PathBuf::from("/test_path/path")),
        };
        let r: RemoteName = s.parse().unwrap();
        assert_eq!(r, a);

        let s = "scp://[::1]:22/test";
        let a = RemoteName {
            user: None,
            host: HostInfo::Ip("::1".parse().unwrap()),
            port: Some(22),
            path: Some(std::path::PathBuf::from("/test")),
        };
        let r: RemoteName = s.parse().unwrap();
        assert_eq!(r, a);

        let s = "scp://localhost";
        let a = RemoteName {
            user: None,
            host: HostInfo::Name("localhost".to_string()),
            port: None,
            path: None,
        };
        let r: RemoteName = s.parse().unwrap();
        assert_eq!(r, a);
    }

    #[test]
    fn test_from_str_with_extra_space() {
        let s = "name@192.168.0.1:  /test_path/path   ";
        let a = RemoteName {
            user: Some("name".to_string()),
            host: HostInfo::Ip("192.168.0.1".parse().unwrap()),
            port: None,
            path: Some(std::path::PathBuf::from("/test_path/path")),
        };
```

```
320             let r: RemoteName = s.parse().unwrap();
321             assert_eq!(r, a);
322 
323             let s = "scp://[::1]:22/  /test";
324             let r: Result<RemoteName, ErrorRemoteName> = s.parse();
325             match r {
326                 Ok(_) => {
327                     unreachable!("この形式はエラーであるはず。");
328                 }
329                 Err(ErrorRemoteName::UriParse(e)) => {
330                     assert_eq!(e.kind(), ParseErrorKind::UnexpectedChar);
331                 }
332                 Err(e) => {
333                     unreachable!("何が起こった?({:?}", e);
334                 }
335             }
336 
337             let s = "scp:// name @192.168.0.1:22/test_path/path";
338             let r: Result<RemoteName, ErrorRemoteName> = s.parse();
339             match r {
340                 Ok(_) => {
341                     unreachable!("この形式はエラーであるはず。");
342                 }
343                 Err(ErrorRemoteName::UriParse(e)) => {
344                     assert_eq!(e.kind(), ParseErrorKind::UnexpectedChar);
345                 }
346                 Err(e) => {
347                     unreachable!("何が起こった?({:?})", e);
348                 }
349             }
350     }
351 }
```

3　ssh2 ログイン処理モジュール ssh_connect.rs

```rust
//! ssh 接続関連関数モジュール

use crate::cmdline_opt::Opt;
use anyhow::{Context, Result, anyhow};
use dialoguer::Password;
use dns_lookup::lookup_host;
use log::{debug, error, info, warn};
use ssh2::Session;
use ssh2_config::{HostParams, ParseRule, SshConfig};
use std::{
    fs::File,
    io::BufReader,
    net::TcpStream,
    path::{Path, PathBuf},
    str,
};

/// ssh 認証で試行するキーファイルの最大数。
/// あまりに、トライ数が多いとサーバー負荷等に影響があるかもしれないので制限する。
const MAX_IDENTITY_TRY: usize = 10;
/// デフォルトのポート番号
const DEFAULT_PORT: u16 = 22;

/// セッションを生成する。
pub fn make_ssh_session(opt: &Opt) -> Result<Session> {
    let host_params = make_host_params(opt).context("Failed to make host parameters.")?;
    let addresses = get_address(&host_params)?;
    let user_name = host_params
        .user
        .as_ref()
        .ok_or(anyhow!("User name is not specified."))?;
    info!(
        "[main] info connection-> user name:\"{}\", ip address:{:?}",
        user_name, addresses
    );
    let identity_file = &host_params.identity_file;

    let ssh = connect_ssh(&addresses[..]).context("The ssh connection failed.")?;
    userauth(&ssh, user_name, identity_file).context("User authentication failed.")?;
    info!("success connect ssh: ip=>{:?}", addresses);
    Ok(ssh)
}

```

```rust
/// ホストパラメータの生成
/// config ファイルより取得したホスト情報をもとに、コマンドラインオプションで上書きしたホストパラメータを生成する。
/// ホスト情報は、コマンドラインオプション>config ファイル>remote_host 引数の順で上書きする。
fn make_host_params(opt: &Opt) -> Result<HostParams> {
    let mut host_params = get_ssh_config(&opt.config_file).query(opt.remote.host.to_string());
    // ホスト名の解決
    if host_params.host_name.is_none() {
        host_params.host_name = Some(opt.remote.host.to_string());
    }
    // ユーザー名の解決
    host_params.user = Some(get_username(opt, &host_params).context("Failed to get user name.")?);
    // 秘密キーファイルの解決
    host_params.identity_file = get_identity_file(opt, &host_params)?;
    // ポート番号の解決
    host_params.port = Some(
        opt.port.unwrap_or(
            host_params
                .port
                .unwrap_or(opt.remote.port.unwrap_or(DEFAULT_PORT)),
        ),
    );
    Ok(host_params)
}

/// ホストの ip アドレス解決
fn get_address(host_params: &HostParams) -> Result<Vec<std::net::SocketAddr>> {
    let dns = host_params
        .host_name
        .as_ref()
        .ok_or(anyhow!("Host name is not specified."))?;
    let port = host_params
        .port
        .ok_or(anyhow!("Port number is not specified."))?;
    let addrs = lookup_host(dns)
        .inspect_err(|e| error!("get_address : Failed lookup_host[{}]", e))
        .context("Cannot find host to connect to.")?
        .map(|addr| std::net::SocketAddr::from((addr, port)))
        .collect::<Vec<_>>();
    if addrs.is_empty() {
        return Err(anyhow!("No address found for the specified host."));
    }
    Ok(addrs)
}

/// ssh-config の取得と解析
```

```rust
/// ファイル名が指定されていない場合は"~/.ssh/config"を使用
/// configファイルのエラー及びファイルがない場合、デフォルト値を返す。
fn get_ssh_config(file_opt: &Option<PathBuf>) -> SshConfig {
    get_config_file(file_opt)
        .map(BufReader::new)
        .map_or(SshConfig::default(), |mut f| {
            SshConfig::default()
                .parse(&mut f, ParseRule::ALLOW_UNKNOWN_FIELDS)
                .unwrap_or_else(|e| {
                    eprintln!("Warning: Failed to parse ssh_config file. Using default settings.
                    ↪ (error: {})", e);
                    SshConfig::default()
                })
        })
}

/// ssh_configファイルがあれば、オープンする。
/// ファイル名の指定がなければ、$Home/.ssh/configを想定する。
fn get_config_file(file_name: &Option<PathBuf>) -> Option<std::fs::File> {
    let file_name = file_name.clone().or_else(|| {
        home::home_dir().map(|p| {
            let mut p = p;
            p.push(".ssh/config");
            p
        })
    });

    file_name.and_then(|p| File::open(p).ok())
}

/// ログイン名を確定し、取得する。
/// ログイン名指定の優先順位は、1. -u引数指定, 2.remote引数, 3.ssh_config指定, 4.現在のユーザー名
fn get_username(opt: &Opt, params: &HostParams) -> Result<String> {
    if let Some(n) = &opt.login_name {
        Ok(n.clone())
    } else if let Some(n) = &opt.remote.user {
        Ok(n.clone())
    } else if let Some(n) = &params.user {
        Ok(n.clone())
    } else if let Some(n) = users::get_current_username() {
        n.to_str()
            .map(|s| s.to_string())
            .ok_or(anyhow!("Invalid login user name. -- {n:?}"))
    } else {
        Err(anyhow!("Could not obtain user name."))
    }
```

```rust
}

/// 秘密キーファイルのパスを取得する
fn get_identity_file(opt: &Opt, host_params: &HostParams) -> Result<Option<Vec<PathBuf>>> {
    if let Some(n) = &opt.identity {
        let path = expand_tilde_in_path(n);
        std::fs::File::open(&path).with_context(|| {
            format!(
                "Unable to access the secret key file specified by the \"-i\" option. [{}]",
                path.to_string_lossy()
            )
        })?;
        Ok(Some(vec![path]))
    } else {
        let name = host_params.identity_file.as_ref();
        match name {
            Some(n) => {
                let paths = n
                    .iter()
                    .map(expand_tilde_in_path)
                    .filter(|p|  match std::fs::File::open(p) {
                        Ok(_) => true,
                        Err(e) => {
                            warn!(
                                "IdentityFile '{:?}' from ssh-config is not accessible. skipping. (io error: {})",
                                p, e
                            );
                            eprintln!(
                                "Warning: IdentityFile '{:?}' from ssh-config is not accessible. skipping.",
                                p
                            );
                            false
                        }
                    })
                    .collect::<Vec<_>>();
                if paths.is_empty() {
                    Err(anyhow!(
                        "No usable identity files found for host {:?} (checked {} entries from ssh-config).",
                        host_params.host_name.as_deref().unwrap_or("<unknown>"),
                        n.len()
                    ))
                } else {
                    Ok(Some(paths))
```

```
                }
            }
            None => Ok(None),
        }
    }
}

// ファイル名の~記号を展開する。
fn expand_tilde_in_path(path: impl AsRef<Path>) -> PathBuf {
    let path_str = path.as_ref().to_string_lossy();
    let expanded_path = shellexpand::tilde(&path_str);
    PathBuf::from(expanded_path.as_ref())
}

/// リモートの ssh に接続し、セッションを生成する。
fn connect_ssh<A: std::net::ToSocketAddrs>(address: A) -> Result<Session> {
    let tcp = TcpStream::connect(address).context("Failed to connect to TCP/IP.")?;
    let mut ssh = Session::new().context("Failed to connect to ssh.")?;
    ssh.set_tcp_stream(tcp);
    ssh.handshake().context("Failed to hanshake ssh.")?;
    Ok(ssh)
}

/// ssh 認証を実施する。
fn userauth(sess: &Session, username: &str, identity: &Option<Vec<PathBuf>>) -> Result<()> {
    if user_auth_agent(sess, username).is_ok() {
        return Ok(());
    }
    if let Some(f) = identity {
        let ret = f
            .iter()
            .take(MAX_IDENTITY_TRY)
            .any(|f| user_auth_identity(sess, username, f).is_ok());
        if ret {
            return Ok(());
        }
    }
    user_auth_password(sess, username)
        .map_err(|_| anyhow!("All user authentication methods failed."))
}

/// agent 認証
fn user_auth_agent(sess: &Session, username: &str) -> Result<(), ssh2::Error> {
    let ret = sess.userauth_agent(username);
    if let Err(e) = &ret {
        debug!("認証失敗 (agent)->{:?}", e);
```

```
223     };
224     ret
225 }
226 
227 /// 公開キー認証
228 fn user_auth_identity(sess: &Session, username: &str, key_file: &Path) -> Result<()> {
229     let mut ret = sess.userauth_pubkey_file(username, None, key_file, None);
230     if ret.is_ok() {
231         return Ok(());
232     };
233     if let ssh2::ErrorCode::Session(-16) = ret.as_ref().unwrap_err().code() {
234         // error_code -16 ->
235         // LIBSSH2_ERROR_FILE:PUBLIC_KEYの取得失敗。多分、秘密キーのパスフレーズ
236         for _i in 0..3 {
237             let password = Password::new()
238                 .with_prompt("Enter the passphrase for the secret key.")
239                 .allow_empty_password(true)
240                 .interact()?;
241             ret = sess.userauth_pubkey_file(username, None, key_file, Some(&password));
242             if ret.is_ok() {
243                 return Ok(());
244             }
245             eprintln!("The passphrase is different.");
246         }
247     }
248     debug!(
249         "Authentication failed(pubkey)->{:?}",
250         ret.as_ref().unwrap_err()
251     );
252     Err(anyhow!("Public key authentication failed."))
253 }
254 
255 /// パスワード認証
256 fn user_auth_password(sess: &Session, username: &str) -> Result<()> {
257     for _i in 0..3 {
258         let password = Password::new()
259             .with_prompt("Enter your login password.")
260             .allow_empty_password(true)
261             .interact()?;
262         let ret = sess.userauth_password(username, &password);
263         if ret.is_ok() {
264             return Ok(());
265         }
266         let ssh2::ErrorCode::Session(-18) = ret.as_ref().unwrap_err().code() else {
267             break;
268         };
```

```
269             // ssh2エラーコード  -18 ->
270             // LIBSSH2_ERROR_AUTHENTICATION_FAILED: パスワードが違うんでしょう。
271             eprintln!("The password is different.");
272             debug!("Authentication failed(password)->{:?}", ret.unwrap_err());
273         }
274         Err(anyhow!("Password authentication failed."))
275     }
276
277     #[cfg(test)]
278     mod test {
279         use super::*;
280         use clap::Parser;
281         #[test]
282         #[ignore]
283         fn make_host_params_test() {
284             let config_file_path = test_config_file_path();
285             let identify = make_dummyidentity_file(1);
286             let opt = make_dummy_opt(format!(
287                 "sshmount -F {} -i {} -p 2223 test_host:/remote/path mnt",
288                 config_file_path.to_string_lossy(),
289                 identify.to_string_lossy()
290             ));
291             let host_param = make_host_params(&opt).unwrap();
292             assert_eq!(host_param.host_name.unwrap(), "example.com");
293             assert_eq!(host_param.port.unwrap(), 2223);
294             assert_eq!(host_param.user.unwrap(), "testuser");
295         }
296
297         #[test]
298         #[ignore]
299         fn test_make_host_params_default_port() {
300             let config_file_path = test_config_file_path();
301             let opt = make_dummy_opt(format!(
302                 "sshmount -F {} default_port:/remote/path mnt",
303                 config_file_path.to_string_lossy(),
304             ));
305             let host_param = make_host_params(&opt).unwrap();
306             assert_eq!(host_param.host_name.unwrap(), "default.example.com");
307             assert_eq!(host_param.port.unwrap(), DEFAULT_PORT);
308             assert_eq!(host_param.user.unwrap(), "defaultuser");
309         }
310
311         #[test]
312         #[ignore]
313         fn test_make_host_params_ip_address_config() {
314             let config_file_path = test_config_file_path();
```

```
315         let opt = make_dummy_opt(format!(
316             "sshmount -F {} 192.168.0.100:/remote/path mnt",
317             config_file_path.to_string_lossy(),
318         ));
319         let host_param = make_host_params(&opt).unwrap();
320         assert_eq!(host_param.host_name.unwrap(), "192.168.0.101");
321         assert_eq!(host_param.port.unwrap(), 2200);
322         assert_eq!(host_param.user.unwrap(), "admin");
323         assert_eq!(
324             host_param.identity_file.unwrap()[0],
325             PathBuf::from("/home/mito/develop/rust/sshmount/test_data/dummy2_rsa")
326         );
327     }
328 
329     #[test]
330     #[ignore]
331     fn test_make_host_params_multi_identify() {
332         let config_file_path = test_config_file_path();
333         let opt = make_dummy_opt(format!(
334             "sshmount -F {} multi_identity:/remote/path mnt",
335             config_file_path.to_string_lossy(),
336         ));
337         let host_param = make_host_params(&opt).unwrap();
338         assert_eq!(host_param.host_name.unwrap(), "multi.example.com");
339         assert_eq!(
340             host_param.identity_file.as_ref().unwrap()[0],
341             PathBuf::from("/home/mito/develop/rust/sshmount/test_data/dummy1_rsa")
342         );
343         assert_eq!(
344             host_param.identity_file.as_ref().unwrap()[1],
345             PathBuf::from("/home/mito/develop/rust/sshmount/test_data/dummy2_rsa")
346         );
347         assert_eq!(host_param.identity_file.as_ref().unwrap().len(), 2);
348     }
349 
350     fn test_config_file_path() -> PathBuf {
351         let d = env!("CARGO_MANIFEST_DIR");
352         let mut p = PathBuf::new();
353         p.push(d);
354         p.push("test_data/config");
355         p
356     }
357 
358     fn make_dummyidentity_file(no: u16) -> PathBuf {
359         let d = env!("CARGO_MANIFEST_DIR");
360         let mut p = PathBuf::new();
```

```
361             p.push(d);
362             p.push(format!("test_data/dummy{}_rsa", no));
363             p
364         }
365 
366         fn make_dummy_opt(cmdline: impl AsRef<str>) -> Opt {
367             let args = cmdline.as_ref().split_whitespace();
368             Opt::try_parse_from(args).unwrap()
369         }
370     }
```

4　FUSE 接続オプション生成モジュール fuse_util.rs

```rust
//! FUSE パラメータ関係　ユーティリティ

use crate::cmdline_opt::Opt;
use anyhow::{Context, Result, ensure};
use ssh2::Session;
use std::env::current_dir;
use std::{
    io::Read,
    path::{Path, PathBuf},
    str,
};

/// マウントポイントのフルパスを生成する
pub fn make_full_path<P: AsRef<Path>>(path: P) -> Result<PathBuf> {
    if path.as_ref().is_absolute() {
        Ok(path.as_ref().to_path_buf())
    } else {
        let mut full_path = current_dir().context("cannot access current directory.")?;
        full_path.push(path);
        Ok(full_path)
    }
}

/// リモート接続先の path の生成
pub fn make_remote_path(opt: &Opt, session: &Session) -> Result<PathBuf> {
    // パスの生成
    const MSG_ERRORHOME: &str = "Fail to generate path name.";
    let mut path = match opt.remote.path {
        Some(ref p) => {
            if p.is_absolute() {
                p.clone()
            } else {
                let mut h = get_home_on_remote(session).context(MSG_ERRORHOME)?;
                h.push(p);
                h
            }
        }
        None => get_home_on_remote(session).context(MSG_ERRORHOME)?,
    };
    // 生成したパスが実在するかを確認する
    let sftp = session
        .sftp()
        .context("Connection to SFTP failed when checking for existence of a path.")?;
```

```
    let file_stat = sftp
        .stat(&path)
        .with_context(|| format!("Cannot find path to connect to. path={:?}", path))?;
    ensure!(
        file_stat.is_dir(),
        "The path to connect to is not a directory."
    );
    // 生成したパスがシンボリックリンクのときは、リンク先を解決する
    let file_stat = sftp
        .lstat(&path)
        .context("Failed to obtain the attributes of the destination directory.")?;
    if file_stat.file_type().is_symlink() {
        path = sftp
            .readlink(&path)
            .context("Failed to resolve symbolic link to connect to.")?;
        if !path.is_absolute() {
            let tmp = path;
            path = get_home_on_remote(session)
                .context("Failed to complete the symbolic link to connect to.")?;
            path.push(tmp);
        };
    };

    Ok(path)
}

/// FUSE の接続時オプションを生成する
pub fn make_mount_option(cmd_opt: &Opt) -> Vec<fuser::MountOption> {
    use fuser::MountOption;

    let mut options = vec![MountOption::FSName("sshfs".to_string())];
    options.push(MountOption::NoDev);
    options.push(MountOption::DirSync);
    options.push(MountOption::Sync);
    options.push(MountOption::DefaultPermissions);
    match cmd_opt.readonly {
        true => options.push(MountOption::RO),
        false => options.push(MountOption::RW),
    }
    match cmd_opt.no_exec {
        true => options.push(MountOption::NoExec),
        false => options.push(MountOption::Exec),
    }
    match cmd_opt.no_atime {
        true => options.push(MountOption::NoAtime),
        false => options.push(MountOption::Atime),
```

```
90      }
91      options
92  }
93
94  /// ssh接続先のカレントディレクトリを取得する
95  fn get_home_on_remote(session: &Session) -> Result<PathBuf> {
96      let mut channel = session
97          .channel_session()
98          .context("Fail to build ssh channel.")?;
99      channel
100         .exec("pwd")
101         .context("Fail to execute \"pwd\" command.")?;
102     let mut buf = Vec::<u8>::new();
103     channel
104         .read_to_end(&mut buf)
105         .context("Fail to get response for \"pwd\" command.")?;
106     channel.close().context("Fail to close ssh channel.")?;
107     str::from_utf8(&buf)
108         .context("The pwd result contains non-utf8 characters.")?
109         .trim()
110         .parse::<PathBuf>()
111         .context("Fail to build path name.")
112 }
```

5 FUSE ファイルシステム本体と関連モジュール

5.1 ファイルシステムモジュール ssh_filesystem.rs

```rust
mod bi_hash_map;
mod file_handle;
mod inode;
mod mutex_lock_ext;

use file_handle::{Dhandles, Fhandles};
use inode::Inodes;
use mutex_lock_ext::MutexLockExt;

use anyhow::Context;
use fuser::{
    BsdFileFlags, Errno, FileAttr, FileHandle, Filesystem, FopenFlags, Generation, INodeNo,
    LockOwner, OpenFlags as OpenFlagsFuser, RenameFlags, ReplyAttr, ReplyData, ReplyDirectory,
    ReplyEntry, Request, WriteFlags,
};
use log::{debug, error, warn};
use ssh2::{ErrorCode, OpenFlags as OpenFlagsSsh2, OpenType, Session, Sftp};
use std::{
    ffi::OsStr,
    io::{Read, Seek, SeekFrom, Write},
    path::{Path, PathBuf},
    sync::Mutex,
    time::{Duration, SystemTime, UNIX_EPOCH},
};

/// FUSE ファイルシステム実装
pub struct Sshfs {
    _session: Session,
    sftp: Sftp,
    inodes: Mutex<Inodes>,
    fhandls: Fhandles,
    dhandls: Dhandles, // opendir時点のディレクトリ一覧を保持する。
    _top_path: PathBuf,
}

impl Sshfs {
    pub fn new<P: AsRef<Path>>(session: Session, path: P) -> anyhow::Result<Self> {
        let mut inodes = Inodes::new();
        let top_path: PathBuf = path.as_ref().into();
        inodes.add(&top_path);
        let inodes = Mutex::new(inodes);
```

```
        let sftp = session
            .sftp()
            .inspect_err(|_| {
                error!("Failed to create sftp from session.");
            })
            .context("Failed to create sftp from session.(Sshfs::new)")?;
        debug!(
            "[Sshfs::new] connect path: <{:?}>, inodes=<{:?}>",
            top_path, inodes
        );
        Ok(Self {
            _session: session,
            sftp,
            inodes,
            fhandls: Fhandles::new(),
            dhandls: Dhandles::new(),
            _top_path: top_path,
        })
    }

    /// ssh2経由でファイルのステータスを取得する。
    /// 副作用: 取得に成功した場合、inodes にパスを登録する。
    fn getattr_from_ssh2(&self, path: &Path, uid: u32, gid: u32) -> Result<FileAttr, Error> {
        let attr_ssh2 = self.sftp.lstat(path)?;
        let kind = Self::conv_file_kind_ssh2fuser(&attr_ssh2.file_type())?;
        let ino = self
            .inodes
            .lock_or_panic("get_attr_from_ssh2", "inodes")
            .add(path);
        Ok(FileAttr {
            ino,
            size: attr_ssh2.size.unwrap_or(0),
            blocks: attr_ssh2.size.unwrap_or(0) / 512 + 1,
            atime: UNIX_EPOCH + Duration::from_secs(attr_ssh2.atime.unwrap_or(0)),
            mtime: UNIX_EPOCH + Duration::from_secs(attr_ssh2.mtime.unwrap_or(0)),
            ctime: UNIX_EPOCH + Duration::from_secs(attr_ssh2.mtime.unwrap_or(0)),
            crtime: UNIX_EPOCH,
            kind,
            perm: attr_ssh2.perm.unwrap_or(0o666) as u16,
            nlink: 1,
            uid,
            gid,
            rdev: 0,
            blksize: 512,
            flags: 0,
        })
```

```
88      }
89
90      fn conv_file_kind_ssh2fuser(filetype: &ssh2::FileType) -> Result<fuser::FileType, Error> {
91          match filetype {
92              ssh2::FileType::NamedPipe => Ok(fuser::FileType::NamedPipe),
93              ssh2::FileType::CharDevice => Ok(fuser::FileType::CharDevice),
94              ssh2::FileType::BlockDevice => Ok(fuser::FileType::BlockDevice),
95              ssh2::FileType::Directory => Ok(fuser::FileType::Directory),
96              ssh2::FileType::RegularFile => Ok(fuser::FileType::RegularFile),
97              ssh2::FileType::Symlink => Ok(fuser::FileType::Symlink),
98              ssh2::FileType::Socket => Ok(fuser::FileType::Socket),
99              ssh2::FileType::Other(_) => Err(Error(Errno::EBADF)),
100         }
101     }
102
103     /// rmdir 失敗時の ssh2 エラーを errno に変換する。
104     /// 一部の SFTP サーバ/ライブラリでは、空でないディレクトリ削除時に
105     /// ENOTEMPTY 相当を直接返さず、曖昧な失敗コードになるため補正する。
106     fn rmdir_errno(e: ssh2::Error) -> Errno {
107         debug!("[rmdir_errno] ssh2::Error => {:?}", e);
108         match e.code() {
109             ErrorCode::Session(-31) | ErrorCode::SFTP(4) => Errno::ENOTEMPTY,
110             _ => Error::from(e).into(),
111         }
112     }
113
114     fn conv_timeornow2systemtime(time: &fuser::TimeOrNow) -> SystemTime {
115         match time {
116             fuser::TimeOrNow::SpecificTime(t) => *t,
117             fuser::TimeOrNow::Now => SystemTime::now(),
118         }
119     }
120 }
121
122 impl Filesystem for Sshfs {
123     fn lookup(&self, req: &Request, parent: INodeNo, name: &OsStr, reply: ReplyEntry) {
124         debug!("[lookup] start: parent={}, name={:?}", parent, name);
125         let Some(mut path) = self
126             .inodes
127             .lock_or_panic("lookup", "inodes")
128             .get_path(parent)
129         else {
130             error!("[lookup] 親ディレクトリの検索に失敗 inode={}", parent);
131             reply.error(Errno::ENOENT);
132             return;
133         };
```

```
134             path.push(Path::new(name));
135             match self.getattr_from_ssh2(&path, req.uid(), req.gid()) {
136                 Ok(attr) => reply.entry(&Duration::from_secs(1), &attr, Generation(0)),
137                 Err(e) => {
138                     reply.error(e.into());
139                 }
140             };
141         }
142 
143         fn getattr(&self, req: &Request, ino: INodeNo, _fh: Option<FileHandle>, reply: ReplyAttr) {
144             debug!("[getattr] start: ino={}", ino);
145             let Some(path) = self.inodes.lock_or_panic("getattr", "inodes").get_path(ino) else {
146                 error!("[getattr] path 取得失敗: inode={}", ino);
147                 reply.error(Errno::ENOENT);
148                 return;
149             };
150             match self.getattr_from_ssh2(&path, req.uid(), req.gid()) {
151                 Ok(attr) => {
152                     //debug!("[getattr]retrun attr: {:?}", &attr);
153                     reply.attr(&Duration::from_secs(1), &attr);
154                 }
155                 Err(e) => {
156                     warn!("[getattr] getattr_from_ssh2 エラー: {:?}", e);
157                     reply.error(e.into())
158                 }
159             };
160         }
161 
162         /// ディレクトリを開く。この時点の一覧を取得し、ハンドルに保持する。
163         /// (修正) 従来は readdir のたびに一覧を取り直していたため、読み出し途中の
164         /// unlink で offset がずれ、remove_dir_all などでエントリが欠落していた。
165         fn opendir(
166             &self,
167             _req: &Request,
168             ino: INodeNo,
169             _flags: OpenFlagsFuser,
170             reply: fuser::ReplyOpen,
171         ) {
172             debug!("[opendir] start: ino={}", ino);
173             let Some(path) = self.inodes.lock_or_panic("opendir", "inodes").get_path(ino) else {
174                 reply.error(Errno::ENOENT);
175                 return;
176             };
177             match self.sftp.readdir(&path) {
178                 Ok(mut dir) => {
179                     let cur_file_attr = ssh2::FileStat {
```

```
                    size: None,
                    uid: None,
                    gid: None,
                    perm: Some(libc::S_IFDIR),
                    atime: None,
                    mtime: None,
                }; // "." ".."の解決用。 attr ディレクトリであることのみを示す。
                dir.insert(0, (Path::new("..").into(), cur_file_attr.clone()));
                dir.insert(0, (Path::new(".").into(), cur_file_attr));
                let fh = FileHandle(self.dhandls.add_dir(dir));
                reply.opened(fh, FopenFlags::empty());
            }
            Err(e) => {
                warn!("[opendir]ssh2::readdir 内でエラー発生-- {:?}", e);
                reply.error(Error::from(e).into());
            }
        }
    }

    fn readdir(
        &self,
        _req: &Request,
        ino: INodeNo,
        fh: FileHandle,
        offset: u64,
        mut reply: ReplyDirectory,
    ) {
        debug!("[readdir] start: ino={}, offset={}", ino, offset);
        // (修正)opendirで保持したスナップショットを使う。
        let Some(dir) = self.dhandls.get_dir(fh.0) else {
            reply.error(Errno::EBADF);
            return;
        };
        for (i, f) in (offset + 1..).zip(dir.iter().skip(offset as usize)) {
            let ino = if f.0 == Path::new("..") || f.0 == Path::new(".") {
                INodeNo(1)
            } else {
                self.inodes.lock_or_panic("readdir", "inodes").add(&f.0)
            };
            let name = match f.0.file_name() {
                Some(n) => n,
                None => f.0.as_os_str(),
            };
            let filetype = &f.1.file_type();
            let filetype = match Self::conv_file_kind_ssh2fuser(filetype) {
                Ok(t) => t,
```

```
                Err(e) => {
                    warn!(
                        "[readdir] ファイルタイプ解析失敗: inode={}, name={:?}",
                        ino, name
                    );
                    reply.error(e.into());
                    return;
                }
            };
            if reply.add(ino, i, filetype, name) {
                break;
            }
        }
        reply.ok();
    }

    /// ディレクトリを閉じ、保持していた一覧を破棄する。
    fn releasedir(
        &self,
        _req: &Request,
        _ino: INodeNo,
        fh: FileHandle,
        _flags: OpenFlagsFuser,
        reply: fuser::ReplyEmpty,
    ) {
        debug!("[releasedir] start: fh={}", fh);
        self.dhandls.del_dir(fh.0);
        reply.ok();
    }

    fn readlink(&self, _req: &Request, ino: INodeNo, reply: ReplyData) {
        debug!("[readlink] start: ino={}", ino);
        let Some(path) = self
            .inodes
            .lock_or_panic("readlink", "inodes")
            .get_path(ino)
        else {
            error!("[readlink] 親ディレクトリの検索に失敗 {ino}");
            reply.error(Errno::ENOENT);
            return;
        };
        match self.sftp.readlink(&path) {
            Ok(p) => {
                //debug!("[readlink] ret_path => {:?}", &p);
                reply.data(p.as_os_str().to_string_lossy().as_bytes());
            }
```

```
272             Err(e) => {
273                 //debug!("[readlink] ssh2::readlink error => {e:?}");
274                 reply.error(Error::from(e).into());
275             }
276         }
277     }
278 
279     fn open(&self, _req: &Request, ino: INodeNo, flags: OpenFlagsFuser, reply: fuser::ReplyOpen) {
280         debug!("[open] start: ino={}, flags={:x}", ino, flags);
281         let Some(file_name) = self.inodes.lock_or_panic("open", "inodes").get_path(ino) else {
282             reply.error(Errno::ENOENT);
283             return;
284         };
285 
286         let mut flags_ssh2 = OpenFlagsSsh2::empty();
287         if flags.0 & libc::O_WRONLY != 0 {
288             flags_ssh2.insert(OpenFlagsSsh2::WRITE);
289         } else if flags.0 & libc::O_RDWR != 0 {
290             flags_ssh2.insert(OpenFlagsSsh2::READ);
291             flags_ssh2.insert(OpenFlagsSsh2::WRITE);
292         } else {
293             flags_ssh2.insert(OpenFlagsSsh2::READ);
294         }
295         if flags.0 & libc::O_APPEND != 0 {
296             flags_ssh2.insert(OpenFlagsSsh2::APPEND);
297         }
298         if flags.0 & libc::O_CREAT != 0 {
299             flags_ssh2.insert(OpenFlagsSsh2::CREATE);
300         }
301         if flags.0 & libc::O_TRUNC != 0 {
302             flags_ssh2.insert(OpenFlagsSsh2::TRUNCATE);
303         }
304         if flags.0 & libc::O_EXCL != 0 {
305             flags_ssh2.insert(OpenFlagsSsh2::EXCLUSIVE);
306         }
307 
308         debug!(
309             "[open] filename='{:?}', openflag = {:?}, bit = {:x}",
310             file_name,
311             flags_ssh2,
312             flags_ssh2.bits()
313         );
314         match self
315             .sftp
316             .open_mode(&file_name, flags_ssh2, 0o777, ssh2::OpenType::File)
317         {
```

```
            Ok(file) => {
                let fh = FileHandle(self.fhandls.add_file(file));
                reply.opened(fh, FopenFlags::empty());
            }
            Err(e) => {
                log::error!(
                    "file-open error: filename='{:?}', mode={:?}, err={}",
                    file_name,
                    flags_ssh2,
                    e
                );
                reply.error(Error::from(e).into());
            }
        }
    }

    fn release(
        &self,
        _req: &Request,
        _ino: INodeNo,
        fh: FileHandle,
        _flags: OpenFlagsFuser,
        _lock_owner: Option<LockOwner>,
        _flush: bool,
        reply: fuser::ReplyEmpty,
    ) {
        debug!("[release] start: fh={}", fh);
        self.fhandls.del_file(fh.0);
        reply.ok();
    }

    fn read(
        &self,
        _req: &Request,
        _ino: INodeNo,
        fh: FileHandle,
        offset: u64,
        size: u32,
        _flags: OpenFlagsFuser,
        _lock_owner: Option<LockOwner>,
        reply: ReplyData,
    ) {
        debug!("[read] start: fh={}, offset={}, size={}", fh, offset, size);
        let Some(file_mutex) = self.fhandls.get_file(fh.0) else {
            reply.error(Errno::EBADF);
            return;
```

```
364         };
365         let mut file = file_mutex.lock_or_panic("read", "file handle");
366 
367         if let Err(e) = file.seek(std::io::SeekFrom::Start(offset)) {
368             reply.error(Error::from(e).into());
369             return;
370         }
371         let mut buff = vec![0; size as usize];
372         let mut read_size: usize = 0;
373         while read_size < size as usize {
374             match file.read(&mut buff[read_size..]) {
375                 Ok(s) => {
376                     if s == 0 {
377                         break;
378                     };
379                     read_size += s;
380                 }
381                 Err(e) => {
382                     reply.error(Error::from(e).into());
383                     return;
384                 }
385             }
386         }
387         buff.resize(read_size, 0u8);
388         reply.data(&buff);
389     }
390 
391     fn write(
392         &self,
393         _req: &Request,
394         _ino: INodeNo,
395         fh: FileHandle,
396         offset: u64,
397         data: &[u8],
398         _write_flags: WriteFlags,
399         _flags: OpenFlagsFuser,
400         _lock_owner: Option<LockOwner>,
401         reply: fuser::ReplyWrite,
402     ) {
403         debug!(
404             "[write] start: fh={}, offset={}, size={}",
405             fh,
406             offset,
407             data.len()
408         );
409         let Some(file_mutex) = self.fhandls.get_file(fh.0) else {
```

```
            reply.error(Errno::EBADF);
            return;
        };
        let file = &mut file_mutex.lock_or_panic("write", "file handle");

        if let Err(e) = file.seek(std::io::SeekFrom::Start(offset)) {
            reply.error(Error::from(e).into());
            return;
        }
        let mut buf = data;
        while !buf.is_empty() {
            let cnt = match file.write(buf) {
                Ok(cnt) => cnt,
                Err(e) => {
                    reply.error(Error::from(e).into());
                    return;
                }
            };
            buf = &buf[cnt..];
        }
        reply.written(data.len() as u32);
    }

    fn lseek(
        &self,
        _req: &Request,
        _ino: INodeNo,
        fh: FileHandle,
        offset: i64,
        whence: i32,
        reply: fuser::ReplyLseek,
    ) {
        debug!(
            "[lseek] start: fh={}, offset={}, whence={}",
            fh, offset, whence
        );
        let seek_from = match whence {
            libc::SEEK_SET => {
                if offset >= 0 {
                    SeekFrom::Start(offset as u64)
                } else {
                    reply.error(Errno::EINVAL);
                    return;
                }
            }
            libc::SEEK_CUR => SeekFrom::Current(offset),
```

```
            libc::SEEK_END => SeekFrom::End(offset),
            _ => {
                reply.error(Errno::EINVAL);
                return;
            }
        };
        let Some(file_mutex) = self.fhandls.get_file(fh.0) else {
            reply.error(Errno::EBADF);
            return;
        };
        let file = &mut file_mutex.lock_or_panic("lseek", "file handle");
        let pos = match file.seek(seek_from) {
            Ok(p) => p,
            Err(e) => {
                reply.error(Error::from(e).into());
                return;
            }
        };
        reply.offset(pos as i64);
    }

    fn mknod(
        &self,
        req: &Request,
        parent: INodeNo,
        name: &OsStr,
        mode: u32,
        umask: u32,
        _rdev: u32,
        reply: ReplyEntry,
    ) {
        debug!(
            "[mknod] start: parent={}, name={:?}, mode={:o}, umask={:o}",
            parent, name, mode, umask
        );
        if mode & libc::S_IFMT != libc::S_IFREG {
            reply.error(Errno::EPERM);
            return;
        }
        let mode = mode & (!umask | libc::S_IFMT);
        let Some(mut new_name) = self
            .inodes
            .lock_or_panic("mknod", "inodes")
            .get_path(parent)
        else {
            reply.error(Errno::ENOENT);
```

```
            return;
        };
        new_name.push(name);
        if let Err(e) = self.sftp.open_mode(
            &new_name,
            OpenFlagsSsh2::CREATE,
            mode as i32,
            OpenType::File,
        ) {
            reply.error(Error::from(e).into());
            return;
        }
        let new_attr = match self.getattr_from_ssh2(&new_name, req.uid(), req.gid()) {
            Ok(a) => a,
            Err(e) => {
                reply.error(e.into());
                return;
            }
        };
        reply.entry(&Duration::from_secs(1), &new_attr, Generation(0));
    }

    fn unlink(&self, _req: &Request, parent: INodeNo, name: &OsStr, reply: fuser::ReplyEmpty) {
        debug!("[unlink] start: parent={}, name={:?}", parent, name);
        let Some(mut path) = self
            .inodes
            .lock_or_panic("unlink", "inodes")
            .get_path(parent)
        else {
            reply.error(Errno::ENOENT);
            return;
        };
        path.push(name);
        match self.sftp.unlink(&path) {
            Ok(_) => {
                self.inodes
                    .lock_or_panic("unlink", "inodes")
                    .del_inode_with_path(&path);
                reply.ok();
            }
            Err(e) => reply.error(Error::from(e).into()),
        }
    }

    fn mkdir(
        &self,
```

```
        req: &Request,
        parent: INodeNo,
        name: &OsStr,
        mode: u32,
        umask: u32,
        reply: ReplyEntry,
    ) {
        debug!(
            "[mkdir] start: parent={}, name={:?}, mode={:o}, umask={:o}",
            parent, name, mode, umask
        );
        let Some(mut path) = self
            .inodes
            .lock_or_panic("mkdir", "inodes")
            .get_path(parent)
        else {
            reply.error(Errno::ENOENT);
            return;
        };
        path.push(name);

        let mode = (mode & (!umask) & 0o777) as i32;

        match self.sftp.mkdir(&path, mode) {
            Ok(_) => match self.getattr_from_ssh2(&path, req.uid(), req.gid()) {
                Ok(attr) => reply.entry(&Duration::from_secs(1), &attr, Generation(0)),
                Err(e) => reply.error(e.into()),
            },
            Err(e) => reply.error(Error::from(e).into()),
        }
    }

    fn rmdir(&self, _req: &Request, parent: INodeNo, name: &OsStr, reply: fuser::ReplyEmpty) {
        debug!("[rmdir] start: parent={}, name={:?}", parent, name);
        let Some(mut path) = self
            .inodes
            .lock_or_panic("mkdir", "inodes")
            .get_path(parent)
        else {
            reply.error(Errno::ENOENT);
            return;
        };
        path.push(name);
        match self.sftp.rmdir(&path) {
            Ok(_) => {
                self.inodes
```

```
                    .lock_or_panic("mkdir", "inodes")
                    .del_inode_with_path(&path);
                reply.ok()
            }
            Err(e) => reply.error(Self::rmdir_errno(e)),
        }
    }

    fn symlink(
        &self,
        req: &Request,
        parent: INodeNo,
        name: &OsStr,
        link: &Path,
        reply: ReplyEntry,
    ) {
        debug!(
            "[symlink] start: parent={}, name={:?}, link={:?}",
            parent, name, link
        );
        let Some(mut target) = self
            .inodes
            .lock_or_panic("symlink", "inodes")
            .get_path(parent)
        else {
            reply.error(Errno::ENOENT);
            return;
        };
        target.push(name);
        match self.sftp.symlink(link, &target) {
            Ok(_) => match self.getattr_from_ssh2(&target, req.uid(), req.gid()) {
                Ok(attr) => reply.entry(&Duration::from_secs(1), &attr, Generation(0)),
                Err(e) => reply.error(e.into()),
            },
            Err(e) => reply.error(Error::from(e).into()),
        }
    }

    fn setattr(
        &self,
        req: &Request,
        ino: INodeNo,
        mode: Option<u32>,
        _uid: Option<u32>,
        _gid: Option<u32>,
        size: Option<u64>,
```

```
        atime: Option<fuser::TimeOrNow>,
        mtime: Option<fuser::TimeOrNow>,
        _ctime: Option<std::time::SystemTime>,
        _fh: Option<FileHandle>,
        _crtime: Option<std::time::SystemTime>,
        _chgtime: Option<std::time::SystemTime>,
        _bkuptime: Option<std::time::SystemTime>,
        _flags: Option<BsdFileFlags>,
        reply: ReplyAttr,
    ) {
        debug!(
            "[setattr] start: ino={}, mode={:?}, size={:?}, atime={:?}, mtime={:?}",
            ino, mode, size, atime, mtime
        );
        let stat = ssh2::FileStat {
            size,
            uid: None,
            gid: None,
            perm: mode,
            atime: atime.map(|t| {
                Self::conv_timeornow2systemtime(&t)
                    .duration_since(UNIX_EPOCH)
                    .unwrap_or_default()
                    .as_secs()
            }),
            mtime: mtime.map(|t| {
                Self::conv_timeornow2systemtime(&t)
                    .duration_since(UNIX_EPOCH)
                    .unwrap_or_default()
                    .as_secs()
            }),
        };
        let Some(filename) = self.inodes.lock_or_panic("setattr", "inodes").get_path(ino) else {
            reply.error(Errno::ENOENT);
            return;
        };
        match self.sftp.setstat(&filename, stat) {
            Ok(_) => {
                let stat = self.getattr_from_ssh2(&filename, req.uid(), req.gid());
                match stat {
                    Ok(s) => reply.attr(&Duration::from_secs(1), &s),
                    Err(e) => reply.error(e.0),
                }
            }
            Err(e) => reply.error(Error::from(e).into()),
        }
```

```
686     }
687
688     fn rename(
689         &self,
690         _req: &Request,
691         parent: INodeNo,
692         name: &OsStr,
693         newparent: INodeNo,
694         newname: &OsStr,
695         flags: RenameFlags,
696         reply: fuser::ReplyEmpty,
697     ) {
698         debug!(
699             "[rename] start: parent={}, name={:?}, newparent={}, newname={:?}, flags={}",
700             parent, name, newparent, newname, flags
701         );
702         let Some(mut old_path) = self
703             .inodes
704             .lock_or_panic("rename", "inodes")
705             .get_path(parent)
706         else {
707             reply.error(Errno::ENOENT);
708             return;
709         };
710         old_path.push(name);
711
712         let Some(mut new_path) = self
713             .inodes
714             .lock_or_panic("rename", "inodes")
715             .get_path(newparent)
716         else {
717             reply.error(Errno::ENOENT);
718             return;
719         };
720         new_path.push(newname);
721
722         // (修正)RENAME_EXCHANGE(入れ替え) は SFTP では実現できない。
723         // 従来は ATOMIC フラグに誤って対応させていた。
724         if flags.contains(RenameFlags::RENAME_EXCHANGE) {
725             reply.error(Errno::EINVAL);
726             return;
727         }
728
729         // (修正) 従来は RENAME_NOREPLACE 指定時にだけ移動先を削除しており、判定が逆だった。
730         // 通常の rename(flags=0) は POSIX では移動先を上書きするが、SFTP の rename は
731         // 移動先が存在すると失敗する (EIO になる)。そのため、移動先を手動で消してから行う。
```

```
        // RENAME_NOREPLACE 指定時は、移動先が存在すれば EEXIST を返す。
        if let Ok(new_stat) = self.sftp.lstat(&new_path) {
            if flags.contains(RenameFlags::RENAME_NOREPLACE) {
                reply.error(Errno::EEXIST);
                return;
            }
            if old_path == new_path {
                reply.ok();
                return;
            }
            let old_stat = match self.sftp.lstat(&old_path) {
                Ok(s) => s,
                Err(e) => {
                    reply.error(Error::from(e).into());
                    return;
                }
            };
            // POSIX の型の組み合わせ規則
            if old_stat.is_dir() && !new_stat.is_dir() {
                reply.error(Errno::ENOTDIR);
                return;
            }
            if !old_stat.is_dir() && new_stat.is_dir() {
                reply.error(Errno::EISDIR);
                return;
            }
            if new_stat.is_dir() {
                // 空でなければ ENOTEMPTY で失敗する (POSIX と同じ)。
                if let Err(e) = self.sftp.rmdir(&new_path) {
                    reply.error(Self::rmdir_errno(e));
                    return;
                }
            } else if let Err(e) = self.sftp.unlink(&new_path) {
                reply.error(Error::from(e).into());
                return;
            }
            self.inodes
                .lock_or_panic("rename", "inodes")
                .del_inode_with_path(&new_path);
        }

        match self
            .sftp
            .rename(&old_path, &new_path, Some(ssh2::RenameFlags::NATIVE))
        {
            Ok(_) => {
```

```
                self.inodes
                    .lock_or_panic("rename", "inodes")
                    .rename(&old_path, &new_path);
                reply.ok();
            }
            Err(e) => reply.error(Error::from(e).into()),
        }
    }
}

#[derive(Debug, Clone, Copy)]
struct Error(Errno);

impl From<Error> for Errno {
    fn from(value: Error) -> Self {
        value.0
    }
}

impl AsRef<Errno> for Error {
    fn as_ref(&self) -> &Errno {
        &self.0
    }
}

impl From<ssh2::Error> for Error {
    fn from(value: ssh2::Error) -> Self {
        let eno = match value.code() {
            ssh2::ErrorCode::Session(_) => Errno::ENXIO,
            ssh2::ErrorCode::SFTP(i) => match i {
                // libssh2の libssh2_sftp.hにて定義されている。
                2 => Errno::ENOENT,         // NO_SUCH_FILE
                3 => Errno::EACCES,         // permission_denied
                4 => Errno::EIO,            // failure
                5 => Errno::ENODEV,         // bad message
                6 => Errno::ENXIO,          // no connection
                7 => Errno::ENETDOWN,       // connection lost
                8 => Errno::ENODEV,         // unsported
                9 => Errno::EBADF,          // invalid handle
                10 => Errno::ENOENT,        //no such path
                11 => Errno::EEXIST,        // file already exists
                12 => Errno::EACCES,        // write protected
                13 => Errno::ENXIO,         // no media
                14 => Errno::ENOSPC,        // no space on filesystem
                15 => Errno::EDQUOT,        // quota exceeded
                16 => Errno::ENODEV,        // unknown principal
```

```
                17 => Errno::ENOLCK,        // lock conflict
                18 => Errno::ENOTEMPTY,     // dir not empty
                19 => Errno::ENOTDIR,       // not a directory
                20 => Errno::ENAMETOOLONG, // invalid file name
                21 => Errno::ELOOP,         // link loop
                _ => {
                    error!("An unknown error occurred during SSH2.[{}]", i);
                    Errno::EIO
                }
            },
        };
        Self(eno)
    }
}

impl From<std::io::Error> for Error {
    fn from(value: std::io::Error) -> Self {
        use std::io::ErrorKind::*;
        let eno = match value.kind() {
            NotFound => Errno::ENOENT,
            PermissionDenied => Errno::EACCES,
            ConnectionRefused => Errno::ECONNREFUSED,
            ConnectionReset => Errno::ECONNRESET,
            ConnectionAborted => Errno::ECONNABORTED,
            NotConnected => Errno::ENOTCONN,
            AddrInUse => Errno::EADDRINUSE,
            AddrNotAvailable => Errno::EADDRNOTAVAIL,
            BrokenPipe => Errno::EPIPE,
            AlreadyExists => Errno::EEXIST,
            WouldBlock => Errno::EWOULDBLOCK,
            InvalidInput => Errno::EINVAL,
            InvalidData => Errno::EILSEQ,
            TimedOut => Errno::ETIMEDOUT,
            WriteZero => Errno::EIO,
            Interrupted => Errno::EINTR,
            Unsupported => Errno::ENOTSUP,
            UnexpectedEof => Errno::EFAULT,
            OutOfMemory => Errno::ENOMEM,
            _ => {
                error!(
                    "An unknown error occurred during std::io.[{}]",
                    value.kind()
                );
                Errno::EIO
            }
        };
```

```
870             Self(eno)
871         }
872     }
```

5.2 ファイルハンドル管理モジュール ssh_filesystem/file_handle.rs

```rust
//! ファイルハンドル管理モジュール

use std::collections::HashMap;
use std::path::PathBuf;
use std::sync::{
    Arc, Mutex,
    atomic::{AtomicU64, Ordering},
};

use super::mutex_lock_ext::MutexLockExt;

/// ファイルハンドル管理構造体
pub(super) struct Fhandles {
    list: Mutex<HashMap<u64, Arc<Mutex<ssh2::File>>>>,
    next_handle: AtomicU64,
}

impl Fhandles {
    pub(super) fn new() -> Self {
        Self {
            list: Mutex::new(HashMap::new()),
            next_handle: AtomicU64::new(0),
        }
    }

    pub(super) fn add_file(&self, file: ssh2::File) -> u64 {
        let handle = self.next_handle.fetch_add(1, Ordering::AcqRel);
        self.list
            .lock_or_panic("fhandles::add_file", "handle list")
            .insert(handle, Arc::new(Mutex::new(file)));
        handle
    }

    pub(super) fn get_file(&self, fh: u64) -> Option<Arc<Mutex<ssh2::File>>> {
        self.list
            .lock_or_panic("fhandles::get_file", "handle list")
            .get(&fh)
            .cloned()
    }

    pub(super) fn del_file(&self, fh: u64) {
        self.list
            .lock_or_panic("fhandles::del_file", "handle list")
            .remove(&fh); // 戻り値は捨てる。この時点でファイルはクローズ。
```

```rust
    }
}

/// opendir 時点のディレクトリ内容 (スナップショット)。"." と ".." を含む。
pub(super) type DirEntries = Vec<(PathBuf, ssh2::FileStat)>;

/// ディレクトリハンドル管理構造体
/// readdir のたびに SFTP から一覧を取り直すと、読み出し途中の unlink 等で
/// offset がずれてエントリが欠落する。そのため、opendir 時の一覧を保持して使い回す。
pub(super) struct Dhandles {
    list: Mutex<HashMap<u64, Arc<DirEntries>>>,
    next_handle: AtomicU64,
}

impl Dhandles {
    pub(super) fn new() -> Self {
        Self {
            list: Mutex::new(HashMap::new()),
            next_handle: AtomicU64::new(0),
        }
    }

    pub(super) fn add_dir(&self, entries: DirEntries) -> u64 {
        let handle = self.next_handle.fetch_add(1, Ordering::AcqRel);
        self.list
            .lock_or_panic("dhandles::add_dir", "dir handle list")
            .insert(handle, Arc::new(entries));
        handle
    }

    pub(super) fn get_dir(&self, fh: u64) -> Option<Arc<DirEntries>> {
        self.list
            .lock_or_panic("dhandles::get_dir", "dir handle list")
            .get(&fh)
            .cloned()
    }

    pub(super) fn del_dir(&self, fh: u64) {
        self.list
            .lock_or_panic("dhandles::del_dir", "dir handle list")
            .remove(&fh);
    }
}
```

5.3 Inode 管理モジュール ssh_filesystem/inode.rs

```rust
//! Inode 管理モジュール

use crate::ssh_filesystem::MutexLockExt;

use super::bi_hash_map::BiHashMap;

use fuser::INodeNo;
use std::path::{Path, PathBuf};
use std::sync::{
    Mutex,
    atomic::{AtomicU64, Ordering},
};

/// Inode 管理構造体
#[derive(Debug, Default)]
pub(super) struct Inodes {
    list: Mutex<BiHashMap<u64, PathBuf>>,
    next_inode: AtomicU64,
}

impl Inodes {
    /// Inodes を生成する
    pub(super) fn new() -> Self {
        Self {
            list: Mutex::new(BiHashMap::new()),
            next_inode: AtomicU64::new(1),
        }
    }

    /// path で指定された inode を生成し、登録する。
    /// すでに path の登録が存在する場合、追加はせず、登録済みの inode を返す。
    /// 初めて、この関数が呼ばれるときは、ファイルシステムにおけるルートであり、inode 番号 1 が割り当てら
    ↪ れる。
    pub(super) fn add<P: AsRef<Path>>(&mut self, path: P) -> INodeNo {
        let mut list_guard = self.list.lock_or_panic("inodes::add", "inode list");
        let path = PathBuf::from(path.as_ref());
        match list_guard.get_left(&path) {
            Some(i) => INodeNo(*i),
            None => {
                let inode = self.next_inode.fetch_add(1, Ordering::AcqRel);
                if list_guard.insert_no_overwrite(inode, path.clone()).is_err() {
                    unreachable!("Unexpected duplicate inode {} or path {:?}", inode, path);
                    // 既に重複がチェックされているので、ありえない。
                }
```

```
                INodeNo(inode)
            }
        }
    }

    /// path から inode を取得する
    /// 主用途が消滅したが、将来のために残しておく
    #[allow(dead_code)]
    pub(super) fn get_inode<P: AsRef<Path>>(&self, path: P) -> Option<INodeNo> {
        let path = PathBuf::from(path.as_ref());
        self.list
            .lock_or_panic("Inodes::get_inode", "inode list")
            .get_left(&path)
            .copied()
            .map(INodeNo)
    }

    /// inode から path を取得する
    pub(super) fn get_path(&self, inode: INodeNo) -> Option<PathBuf> {
        self.list
            .lock_or_panic("Inodes::get_path", "inode list")
            .get_right(&inode.into())
            .cloned()
    }

    /// inodes から、inode の登録を削除する
    /// (主用途がなくなっちゃったけど、将来のために残しておく)
    #[allow(dead_code)]
    pub(super) fn del_inode(&mut self, inode: INodeNo) -> Option<INodeNo> {
        self.list
            .lock_or_panic("Inodes::del_inode", "inode list")
            .remove_left(&inode.into())
            .map(|_| inode)
    }

    /// path 名から iNode の登録を削除する
    pub(super) fn del_inode_with_path<P: AsRef<Path>>(&mut self, path: P) -> Option<INodeNo> {
        let path = PathBuf::from(path.as_ref());
        self.list
            .lock_or_panic("Inodes::del_inode_with_path", "inode list")
            .remove_right(&path)
            .map(INodeNo)
    }

    /// 登録されている inode の path を変更する。
    /// old_path が存在しなければ、なにもしない。
```

```
90      pub(super) fn rename<P: AsRef<Path>>(&mut self, old_path: P, new_path: P) {
91          let old_path = PathBuf::from(old_path.as_ref());
92          let new_path = PathBuf::from(new_path.as_ref());
93          let mut list_guard = self.list.lock_or_panic("Inodes::rename", "inode list");
94          let Some(ino) = list_guard.get_left(&old_path).copied() else {
95              return;
96          };
97          list_guard.remove_left(&ino);
98          list_guard.insert(ino, new_path.clone());
99          // ディレクトリ配下のファイルのパスを変更する。
100         // old_pathがファイルだったとしても、strip_prefixが全件一致しないだけだから問題なし。
101         // old_path本体は、すでに変更されているので、ここでは一致しない。
102         let need_update_files: Vec<(u64, PathBuf)> = list_guard
103             .iter()
104             .filter_map(|(ino, path)| {
105                 let suffix = path.strip_prefix(&old_path).ok()?;
106                 Some((*ino, new_path.join(suffix)))
107             })
108             .collect();
109         for (ino, new_path) in need_update_files {
110             list_guard.remove_left(&ino);
111             list_guard.insert(ino, new_path);
112         }
113     }
114 }
115
116 #[cfg(test)]
117 mod inode_test {
118     use super::Inodes;
119     use fuser::INodeNo;
120     use std::path::Path;
121
122     #[test]
123     fn inode_add_test() {
124         let mut inodes = Inodes::new();
125         assert_eq!(inodes.add(""), INodeNo(1));
126         assert_eq!(inodes.add(Path::new("test")), INodeNo(2));
127         assert_eq!(inodes.add(Path::new("")), INodeNo(1));
128         assert_eq!(inodes.add(Path::new("test")), INodeNo(2));
129         assert_eq!(inodes.add(Path::new("test3")), INodeNo(3));
130         assert_eq!(inodes.add(Path::new("/test")), INodeNo(4));
131         assert_eq!(inodes.add(Path::new("test/")), INodeNo(2));
132     }
133
134     fn make_inodes() -> Inodes {
135         let mut inodes = Inodes::new();
```

```
136             inodes.add(Path::new(""));
137             inodes.add(Path::new("test"));
138             inodes.add(Path::new("test2"));
139             inodes.add(Path::new("test3/"));
140             inodes
141         }
142
143         #[test]
144         fn inodes_get_inode_test() {
145             let inodes = make_inodes();
146             assert_eq!(inodes.get_inode(Path::new("")), Some(INodeNo(1)));
147             assert_eq!(inodes.get_inode(Path::new("test4")), None);
148             assert_eq!(inodes.get_inode(Path::new("/test")), None);
149             assert_eq!(inodes.get_inode(Path::new("test3")), Some(INodeNo(4)));
150         }
151
152         #[test]
153         fn inodes_get_path_test() {
154             let inodes = make_inodes();
155             assert_eq!(inodes.get_path(INodeNo(1)), Some(Path::new("").into()));
156             assert_eq!(inodes.get_path(INodeNo(3)), Some(Path::new("test2").into()));
157             assert_eq!(inodes.get_path(INodeNo(5)), None);
158             assert_eq!(
159                 inodes.get_path(INodeNo(3)),
160                 Some(Path::new("test2/").into())
161             );
162         }
163
164         #[test]
165         fn inodes_rename() {
166             let mut inodes = make_inodes();
167             let old = Path::new("test2");
168             let new = Path::new("new_test");
169             let ino = inodes.get_inode(old).unwrap();
170             inodes.rename(old, new);
171             assert_eq!(inodes.get_path(ino), Some(new.into()));
172
173             let mut inodes = make_inodes();
174             let inodes2 = make_inodes();
175             inodes.rename(Path::new("nai"), Path::new("kawattenai"));
176             assert_eq!(*inodes.list.lock().unwrap(), *inodes2.list.lock().unwrap());
177         }
178
179         #[test]
180         fn inodes_rename_dir() {
181             let mut inodes = make_inodes();
```

```
182         let old_dir = Path::new("test3");
183         let dir_inode = inodes.get_inode(old_dir).unwrap();
184         let inode1 = inodes.add(old_dir.join("file1"));
185         let inode2 = inodes.add(old_dir.join("file2"));
186         let new_dir = Path::new("new_test3");
187         inodes.rename(old_dir, new_dir);
188         assert_eq!(
189             inodes.get_path(dir_inode),
190             Some(new_dir.into()),
191             "ディレクトリの inode のパスがおかしい"
192         );
193         let new_file1 = inodes.get_path(inode1).unwrap();
194         let new_file2 = inodes.get_path(inode2).unwrap();
195         assert_eq!(
196             new_file1,
197             new_dir.join("file1"),
198             "ファイル 1 のパスがおかしい"
199         );
200         assert_eq!(
201             new_file2,
202             new_dir.join("file2"),
203             "ファイル 2 のパスがおかしい"
204         );
205         assert_eq!(
206             inodes.get_inode(new_dir.join("file1")),
207             Some(inode1),
208             "逆引きした inode が一致しない"
209         );
210         assert_eq!(
211             inodes.get_inode(new_dir.join("file2")),
212             Some(inode2),
213             "逆引きした inode が一致しない"
214         );
215         assert_eq!(
216             inodes.get_inode(old_dir.join("file1")),
217             None,
218             "古いパスで逆引きできてしまう"
219         );
220         assert_eq!(
221             inodes.get_inode(old_dir.join("file2")),
222             None,
223             "古いパスで逆引きできてしまう"
224         );
225     }
226 }
```

5.4 双方向ハッシュマップモジュール ssh_filesystem/bi_hash_map.rs

```rust
//! bidirectional hash map
//! 双方向ハッシュマップ

use std::{collections::HashMap, hash::Hash, sync::Arc};

/// 双方向ハッシュマップ
#[derive(Debug, Default, PartialEq, Eq)]
pub(super) struct BiHashMap<L, R>
where
    L: Hash + Eq + Clone,
    R: Hash + Eq + Clone,
{
    left: HashMap<Arc<L>, Arc<R>>,
    right: HashMap<Arc<R>, Arc<L>>,
}

impl<L, R> BiHashMap<L, R>
where
    L: Hash + Eq + Clone,
    R: Hash + Eq + Clone,
{
    /// 新しい双方向ハッシュマップを生成する
    pub fn new() -> Self {
        BiHashMap {
            right: HashMap::new(),
            left: HashMap::new(),
        }
    }

    /// チェック無しで挿入する。
    /// この時点で与えられる引数は、R,L のいずれも既存のキーと重複しないことが保証されている必要がある。
    fn insert_no_check(&mut self, left: L, right: R) {
        let right = Arc::new(right);
        let left = Arc::new(left);
        self.right.insert(right.clone(), left.clone());
        self.left.insert(left, right);
    }

    /// マップに新しい要素を挿入する。
    /// 既存のキーと重複する場合、対応する値を上書きし、OverwriteResult で通知する。
    pub fn insert(&mut self, left: L, right: R) -> OverwriteResult<L, R> {
        let old_left = self.right.remove(&right);
        let old_right = self.left.remove(&left);
        let result = match (old_left, old_right) {
```

```
            (None, None) => OverwriteResult::NoOverwrite,
            (Some(old_l), None) => {
                self.left.remove(old_l.as_ref());
                OverwriteResult::OverwriteLeft((*old_l).clone())
            }
            (None, Some(old_r)) => {
                self.right.remove(old_r.as_ref());
                OverwriteResult::OverwriteRight((*old_r).clone())
            }
            (Some(old_l), Some(old_r)) => {
                self.left.remove(old_l.as_ref());
                self.right.remove(old_r.as_ref());
                OverwriteResult::OverwriteBoth(
                    (left.clone(), (*old_r).clone()),
                    ((*old_l).clone(), right.clone()),
                )
            }
        };
        self.insert_no_check(left, right);
        result
    }

    /// マップに新しい値を挿入する
    /// 既存のキーが存在する場合は、エラーを返す。
    pub fn insert_no_overwrite(&mut self, left: L, right: R) -> Result<(), ()> {
        if self.contains_left(&left) || self.contains_right(&right) {
            return Err(());
        }
        self.insert_no_check(left, right);
        Ok(())
    }

    /// 左側のキーから右側の値を取得する
    /// 存在しない場合は `None` を返す
    pub fn get_right(&self, left: &L) -> Option<&R> {
        self.left.get(left).map(|arc_r| arc_r.as_ref())
    }

    /// 右側のキーから左側の値を取得する
    /// 存在しない場合は `None` を返す
    pub fn get_left(&self, right: &R) -> Option<&L> {
        self.right.get(right).map(|arc_l| arc_l.as_ref())
    }

    /// 左側のキーが存在するかどうかを返す
    /// 存在する場合は `true`、存在しない場合は `false` を返す
```

```rust
    pub fn contains_left(&self, left: &L) -> bool {
        self.left.contains_key(left)
    }

    /// 右側のキーが存在するかどうかを返す
    /// 存在する場合は *true*、存在しない場合は *false* を返す
    pub fn contains_right(&self, right: &R) -> bool {
        self.right.contains_key(right)
    }

    /// 左側の値から、リストの項目を削除する
    /// 存在しない場合は、なにもしない。
    /// 以前の値を返す。(存在しない場合は *None*)
    pub fn remove_left(&mut self, left: &L) -> Option<R> {
        let result = self.left.remove(left).map(|arc_r| (*arc_r).clone());
        if let Some(right) = result.as_ref() {
            self.right.remove(right);
        };
        result
    }

    /// 右側の値から、リストの項目を削除する
    /// 存在しない場合は、なにもしない。
    /// 以前の値を返す。(存在しない場合は *None*)
    pub fn remove_right(&mut self, right: &R) -> Option<L> {
        let result = self.right.remove(right).map(|arc_l| (*arc_l).clone());
        if let Some(left) = result.as_ref() {
            self.left.remove(left);
        };
        result
    }

    /// リスト内全件の列挙を返す。
    /// 返却されるタプルの順序は保証しない。
    pub fn iter(&self) -> impl Iterator<Item = (&L, &R)> {
        self.left.iter().map(|(l, r)| (l.as_ref(), r.as_ref()))
    }
}

/// 挿入時の上書き結果
#[derive(Debug, PartialEq, Eq)]
pub enum OverwriteResult<L, R> {
    NoOverwrite,
    OverwriteRight(R),
    OverwriteLeft(L),
    OverwriteBoth((L, R), (L, R)),
```

```rust
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    /// 単純な挿入と、値の取得のテスト
    fn tanjyunna_insert() {
        let mut bimap = BiHashMap::new();
        assert_eq!(bimap.insert(1, "a"), OverwriteResult::NoOverwrite);
        assert_eq!(bimap.get_right(&1), Some(&"a"));
        assert_eq!(bimap.get_left(&"a"), Some(&1));
        assert_eq!(bimap.insert_no_overwrite(2, "b"), Ok(()));
        assert_eq!(bimap.get_right(&2), Some(&"b"));
        assert_eq!(bimap.get_left(&"b"), Some(&2));
        assert!(bimap.contains_left(&1));
        assert!(bimap.contains_right(&"b"));
    }

    #[test]
    /// 上書き挿入のテスト
    /// 左側、右側、両側の上書きのケースを確認する
    fn overwrite_insert() {
        let mut bimap = BiHashMap::new();
        assert_eq!(bimap.insert(1, "a"), OverwriteResult::NoOverwrite);
        print_hash_map(&bimap, "insert (1, 'a')");
        assert_eq!(bimap.insert(1, "b"), OverwriteResult::OverwriteRight("a"));
        print_hash_map(&bimap, "insert (1, 'b')");
        assert_eq!(bimap.get_right(&1), Some(&"b"));
        assert_eq!(bimap.insert(2, "b"), OverwriteResult::OverwriteLeft(1));
        print_hash_map(&bimap, "insert (2, 'b')");
        assert_eq!(bimap.get_left(&"b"), Some(&2));
        assert_eq!(bimap.get_right(&2), Some(&"b"));
        assert_eq!(bimap.insert_no_overwrite(3, "c"), Ok(()));
        print_hash_map(&bimap, "insert (3, 'c')");
        assert_eq!(
            bimap.insert(2, "b"),
            OverwriteResult::OverwriteBoth((2, "b"), (2, "b"))
        );
        print_hash_map(&bimap, "insert (2, 'b') again");
        assert_eq!(
            bimap.insert(3, "b"),
            OverwriteResult::OverwriteBoth((3, "c"), (2, "b"))
        );
        print_hash_map(&bimap, "insert (3, 'b')");
```

```
183         assert_eq!(bimap.get_right(&3), Some(&"b"));
184         assert_eq!(bimap.get_left(&"b"), Some(&3));
185         assert_eq!(bimap.left.len(), bimap.right.len());
186         assert_eq!(bimap.get_right(&2), None);
187         assert_eq!(bimap.insert_no_overwrite(3, "d"), Err(()));
188         print_hash_map(&bimap, "insert_no_overwright (3, 'd') [fail]");
189         assert_eq!(bimap.insert_no_overwrite(5, "e"), Ok(()));
190         print_hash_map(&bimap, "insert_no_overwright (5, 'e') [ok]");
191         assert_eq!(bimap.insert_no_overwrite(5, "d"), Err(()));
192         print_hash_map(&bimap, "insert_no_overwright (5, 'd') [fail]");
193     }
194 
195     /// 左右の削除のテスト
196     #[test]
197     fn remove_test() {
198         let mut bimap = BiHashMap::new();
199         bimap.insert_no_check(1, "a");
200         bimap.insert_no_check(2, "b");
201         bimap.insert_no_check(3, "c");
202         print_hash_map(&bimap, "initial map");
203         assert_eq!(bimap.remove_left(&2), Some("b"));
204         print_hash_map(&bimap, "after remove_left(2)");
205         assert_eq!(bimap.get_left(&"b"), None);
206         assert_eq!(bimap.get_right(&2), None);
207         assert_eq!(bimap.remove_right(&"c"), Some(3));
208         print_hash_map(&bimap, "after remove_right('c')");
209         assert_eq!(bimap.get_left(&"c"), None);
210         assert_eq!(bimap.get_right(&3), None);
211         assert_eq!(bimap.remove_left(&4), None);
212         print_hash_map(&bimap, "after remove_left(4) [no op]");
213         assert_eq!(bimap.remove_right(&"d"), None);
214         print_hash_map(&bimap, "after remove_right('d') [no op]");
215     }
216 
217     use std::fmt::Debug;
218     fn print_hash_map<R, L>(bimap: &BiHashMap<R, L>, mes: &str)
219     where
220         R: Debug + Hash + Eq + Clone,
221         L: Debug + Hash + Eq + Clone,
222     {
223         println!("=== {} ===", mes);
224         println!("Left to Right:");
225         for (l, r) in &bimap.left {
226             println!("  {:?} => {:?}", l, r);
227         }
228         println!("Right to Left:");
```

```
229         for (r, l) in &bimap.right {
230             println!("  {:?} => {:?}", r, l);
231         }
232     }
233 }
```

5.5 Mutex 用拡張モジュール ssh_filesystem/mutex_lock_ext.rs

```rust
use log::error;
use std::sync::{Mutex, MutexGuard};

/// Mutex の独自拡張
pub(super) trait MutexLockExt<T> {
    /// Mutex をロックし、毒化されていた場合は、ログを取得した上で即座に panic する。
    fn lock_or_panic(&self, func: impl AsRef<str>, item: impl AsRef<str>) -> MutexGuard<'_, T>;
}

impl<T> MutexLockExt<T> for Mutex<T> {
    fn lock_or_panic(&self, func: impl AsRef<str>, item: impl AsRef<str>) -> MutexGuard<'_, T> {
        match self.lock() {
            Ok(guard) => guard,
            Err(_poisoned) => {
                let func = func.as_ref();
                let item = item.as_ref();
                error!(
                    "{}: 致命的エラー。{}の Mutex が毒化された。panic!() する。",
                    func, item
                );
                panic!("{}: 致命的エラー。{}の Mutex が毒化された。", func, item);
            }
        }
    }
}
```

6　統合テスト用モジュール関係

6.1　単純なファイルアクセスのテストモジュール tests/simple_file_access.rs

```rust
//! `sshmount` を実際に起動して、リモート上のファイルアクセスを検証する統合テスト。
//!
//! このテストは `cargo make test` での実行を前提とする。
//! 環境変数は `test/test.env` から `cargo-make` により読み込まれ、
//! テスト前に `sshmount` のビルド・接続準備・起動、テスト後に接続解放が行われる。
//!
//! 外部環境に依存するため、通常の `cargo test` では実行せず、各テストは `#[ignore]` を付けて運用する。
mod common;

use std::{
    fs::{self},
    io::Write,
    path::Path,
    sync::{Arc, Barrier},
};

use common::get_test_root_dir;

/// [01] ファイルを作成し、存在を確認し、削除して、存在しないことを確認する。
#[test]
#[ignore]
fn test_create_and_delete_file() -> Result<(), Box<dyn std::error::Error>> {
    let root_dir = get_test_root_dir()?;
    let branch_num = uuid::Uuid::now_v7().to_string();
    let test_file_path = root_dir.join(format!("test_01_{}", branch_num));
    assert!(
        !test_file_path.try_exists()?,
        "まだ作ってないファイルが存在する。"
    );
    fs::File::create(&test_file_path)?;
    assert!(
        test_file_path.try_exists()?,
        "作成したファイルが存在しない。"
    );
    fs::remove_file(&test_file_path)?;
    assert!(
        !test_file_path.try_exists()?,
        "削除したファイルが存在する。"
    );
    Ok(())
}
```

```rust

/// [02] ファイルを作成し、ファイル名に使った UUID を保存。ファイルを開き直して、UUID を読み出して一致を
↪ 確認する。
#[test]
#[ignore]
fn test_write_and_read_file() -> Result<(), Box<dyn std::error::Error>> {
    let root_dir = get_test_root_dir()?;
    let branch_num = uuid::Uuid::now_v7().to_string();
    let test_file_path = root_dir.join(format!("test_02_{}", branch_num));
    {
        let mut file = fs::File::create(&test_file_path)?;
        use std::io::Write;
        file.write_all(branch_num.as_bytes())?;
    }
    {
        let content = fs::read_to_string(&test_file_path)?;
        assert_eq!(
            content, branch_num,
            "書き込んだ内容と読み出した内容が一致しない。"
        );
    }
    fs::remove_file(&test_file_path)?;
    Ok(())
}

/// [03] ファイルを作成し、rename して、一応内容も確認する
#[test]
#[ignore]
fn test_rename_file() -> Result<(), Box<dyn std::error::Error>> {
    let root_dir = get_test_root_dir()?;
    let branch_num = uuid::Uuid::now_v7().to_string();
    let test_file_path = root_dir.join(format!("test_03_{}", branch_num));
    let renamed_file_path = root_dir.join(format!("test_03_renamed_{}", branch_num));
    {
        let mut file = fs::File::create(&test_file_path)?;
        use std::io::Write;
        file.write_all(branch_num.as_bytes())?;
    }
    fs::rename(&test_file_path, &renamed_file_path)?;
    {
        let content = fs::read_to_string(&renamed_file_path)?;
        assert_eq!(
            content, branch_num,
            "書き込んだ内容と読み出した内容が一致しない。"
        );
    }
```

```rust
    fs::remove_file(&renamed_file_path)?;
    Ok(())
}

/// [04] ディレクトリの作成と削除
/// - ディレクトリを作成し、存在の確認
/// - デイレクトリ内にファイルを作成し、内容の確認
/// - ここでディレクトリを削除してみて、エラーになることを確認
/// - ディレクトリ内のファイルを削除
/// - ディレクトリを削除し、不存在を確認
#[test]
#[ignore]
fn test_create_and_delete_dir() -> Result<(), Box<dyn std::error::Error>> {
    let root_dir = get_test_root_dir()?;
    let dir_branch_num = uuid::Uuid::now_v7().to_string();
    let test_dir_path = root_dir.join(format!("test_04_dir_{}", dir_branch_num));
    let file_branch_num = uuid::Uuid::now_v7().to_string();
    let test_file_path = test_dir_path.join(format!("test_04_file_{}", file_branch_num));
    // ディレクトリの作成と確認
    fs::create_dir(&test_dir_path)?;
    assert!(
        test_dir_path.try_exists()?,
        "作成したディレクトリが存在しない。"
    );
    assert!(
        test_dir_path.is_dir(),
        "作成したものがディレクトリではない。"
    );
    // ファイルの作成と書き込み
    {
        let mut file = fs::File::create(&test_file_path)?;
        file.write_all(file_branch_num.as_bytes())?;
    }
    {
        let content = fs::read_to_string(&test_file_path)?;
        assert_eq!(
            content, file_branch_num,
            "書き込んだ内容と読み出した内容が一致しない。"
        );
    }
    // ディレクトリの削除失敗の確認
    let remove_dir_err = fs::remove_dir(&test_dir_path).expect_err("削除できちゃだめ^^;");
    assert_eq!(
        remove_dir_err.kind(),
        std::io::ErrorKind::DirectoryNotEmpty,
        "エラー内容が想定外 [{}]",
```

```
133             remove_dir_err
134         );
135         // ディレクトリの削除は中身を消してからね。
136         fs::remove_file(&test_file_path)?;
137         fs::remove_dir(&test_dir_path)?;
138         assert!(
139             !test_dir_path.try_exists()?,
140             "削除したディレクトリが存在する。"
141         );
142         Ok(())
143     }
144
145     /// [05] ディレクトリの rename
146     /// ディレクトリの名前を変更し、その配下のファイルの内容も確認する。
147     #[test]
148     #[ignore]
149     fn test_rename_dir() -> Result<(), Box<dyn std::error::Error>> {
150         let root_dir = get_test_root_dir()?;
151         let dir_branch_num = uuid::Uuid::now_v7().to_string();
152         let test_dir_path = root_dir.join(format!("test_05_dir_{}", dir_branch_num));
153         let renamed_dir_path = root_dir.join(format!("test_05_renamed_dir_{}", dir_branch_num));
154         let file_branch_num = uuid::Uuid::now_v7().to_string();
155         let test_file_path = test_dir_path.join(format!("test_05_file_{}", file_branch_num));
156         let renamed_file_path = renamed_dir_path.join(format!("test_05_file_{}", file_branch_num));
157
158         fs::create_dir(&test_dir_path)?;
159         {
160             let mut file = fs::File::create(&test_file_path)?;
161             file.write_all(file_branch_num.as_bytes())?;
162         }
163         fs::rename(&test_dir_path, &renamed_dir_path)?;
164         assert!(!test_dir_path.try_exists()?, "元のディレクトリが存在する。");
165         assert!(
166             renamed_dir_path.try_exists()?,
167             "リネーム後のディレクトリが存在しない。"
168         );
169         {
170             assert!(
171                 renamed_file_path.try_exists()?,
172                 "リネーム後のディレクトリ内のファイルが存在しない。"
173             );
174             let content = fs::read_to_string(&renamed_file_path)?;
175             assert_eq!(
176                 content, file_branch_num,
177                 "書き込んだ内容と読み出した内容が一致しない。"
178             );
```

```
    }
    fs::remove_dir_all(renamed_dir_path)?;
    Ok(())
}

/// [06] ディレクトリ内にディレクトリがあるときの rename の挙動
#[test]
#[ignore]
fn test_rename_dir_with_subdir() -> Result<(), Box<dyn std::error::Error>> {
    let root_dir = get_test_root_dir()?;
    let dir_branch_num = uuid::Uuid::now_v7().to_string();
    let test_dir_path = root_dir.join(format!("test_06_dir_{}", dir_branch_num));
    let test_sub_dir_branch_num = uuid::Uuid::now_v7().to_string();
    let test_sub_dir_path =
        test_dir_path.join(format!("test_06_subdir_{}", test_sub_dir_branch_num));
    let test_file_branch_num = uuid::Uuid::now_v7().to_string();
    let test_file_path = test_sub_dir_path.join(format!("test_06_file_{}", test_file_branch_num));
    let renamed_dir_path = root_dir.join(format!("test_06_renamed_dir_{}", dir_branch_num));
    let renamed_sub_dir_path_sufix = test_sub_dir_path.strip_prefix(&test_dir_path).unwrap();
    let renamed_sub_dir_path = renamed_dir_path.join(renamed_sub_dir_path_sufix);
    let renamed_file_path_sufix = test_file_path.strip_prefix(&test_dir_path).unwrap();
    let renamed_file_path = renamed_dir_path.join(renamed_file_path_sufix);

    fs::create_dir(&test_dir_path)?;
    fs::create_dir(&test_sub_dir_path)?;
    {
        let mut file = fs::File::create(&test_file_path)?;
        file.write_all(test_file_branch_num.as_bytes())?;
    }

    assert!(
        test_file_path.try_exists()?,
        "作成したファイルが存在しない。"
    );

    fs::rename(&test_dir_path, &renamed_dir_path)?;

    assert!(
        renamed_sub_dir_path.try_exists()?,
        "リネーム後のディレクトリ内のサブディレクトリが存在しない。"
    );
    assert!(
        renamed_file_path.try_exists()?,
        "リネーム後のディレクトリ内のファイルが存在しない。"
    );
    {
```

```
225         let content = fs::read_to_string(&renamed_file_path)?;
226         assert_eq!(
227             content, test_file_branch_num,
228             "書き込んだ内容と読み出した内容が一致しない。"
229         );
230     }
231 
232     fs::remove_dir_all(&renamed_dir_path)?;
233     Ok(())
234 }
235 
236 /// [07] 変更後のディレクトリ名が既存のディレクトリ (空) と同じ場合の挙動
237 #[test]
238 #[ignore]
239 fn test_rename_dir_to_existing_empty_dir() -> Result<(), Box<dyn std::error::Error>> {
240     let root_dir = get_test_root_dir()?;
241     let dir_branch_num = uuid::Uuid::now_v7().to_string();
242     let test_dir_path = root_dir.join(format!("test_07_dir_{}", dir_branch_num));
243     let exist_dir_path = root_dir.join(format!("test_07_exist_dir_{}", dir_branch_num));
244 
245     fs::create_dir(&test_dir_path)?;
246     fs::create_dir(&exist_dir_path)?;
247 
248     fs::rename(&test_dir_path, &exist_dir_path).expect("リネーム先が空なので成功するはず。");
249     assert!(!test_dir_path.try_exists()?, "元のディレクトリが存在する。");
250     assert!(
251         exist_dir_path.try_exists()?,
252         "リネーム後のディレクトリが存在しない。"
253     );
254     fs::remove_dir(&exist_dir_path)?;
255 
256     Ok(())
257 }
258 
259 /// [08] 変更後のディレクトリ名が既存のディレクトリ (空でない) と同じ場合の挙動
260 #[test]
261 #[ignore]
262 fn test_rename_dir_to_existing_nonempty_dir() -> Result<(), Box<dyn std::error::Error>> {
263     let root_dir = get_test_root_dir()?;
264     let dir_branch_num = uuid::Uuid::now_v7().to_string();
265     let test_dir_path = root_dir.join(format!("test_08_dir_{}", dir_branch_num));
266     let exist_dir_path = root_dir.join(format!("test_08_exist_dir_{}", dir_branch_num));
267     let exist_dir_file_path = exist_dir_path.join(format!("test_08_exist_file_{}",
    ↪  dir_branch_num));
268 
269     fs::create_dir(&test_dir_path)?;
```

```
    fs::create_dir(&exist_dir_path)?;
    {
        let mut file = fs::File::create(&exist_dir_file_path)?;
        file.write_all(dir_branch_num.as_bytes())?;
    }

    let rename_err = fs::rename(&test_dir_path, &exist_dir_path)
        .expect_err("リネーム先が空でないので失敗するはず。");
    assert_eq!(
        rename_err.kind(),
        std::io::ErrorKind::DirectoryNotEmpty,
        "エラー内容が想定外 [{}]",
        rename_err
    );

    fs::remove_dir(&test_dir_path)?;
    fs::remove_dir_all(&exist_dir_path)?;
    Ok(())
}

/// [09] ファイルのディレクトリ間ムーブの挙動
#[test]
#[ignore]
fn test_move_file_between_dirs() -> Result<(), Box<dyn std::error::Error>> {
    let root_dir = get_test_root_dir()?;
    let dir_branch_num = uuid::Uuid::now_v7().to_string();
    let from_dir_path = root_dir.join(format!("test_09_from_dir_{}", dir_branch_num));
    let to_dir_path = root_dir.join(format!("test_09_to_dir_{}", dir_branch_num));
    let test_file_path = from_dir_path.join(format!("test_09_file_{}", dir_branch_num));
    let moved_file_path = to_dir_path.join(format!("test_09_file_{}", dir_branch_num));

    fs::create_dir(&from_dir_path)?;
    fs::create_dir(&to_dir_path)?;
    {
        let mut file = fs::File::create(&test_file_path)?;
        file.write_all(dir_branch_num.as_bytes())?;
    }

    fs::rename(&test_file_path, &moved_file_path)?;
    assert!(!test_file_path.try_exists()?, "元のファイルが存在する。");
    assert!(
        moved_file_path.try_exists()?,
        "移動後のファイルが存在しない。"
    );
    {
        let content = fs::read_to_string(&moved_file_path)?;
```

```
316         assert_eq!(
317             content, dir_branch_num,
318             "書き込んだ内容と読み出した内容が一致しない。"
319         );
320     }
321 
322     fs::remove_dir(&from_dir_path)?;
323     fs::remove_dir_all(&to_dir_path)?;
324 
325     Ok(())
326 }
327 
328 /// [10] ディレクトリ間のディレクトリのムーブの挙動。
329 /// ちゃんと、配下のファイルも連れて行ってくれること。
330 #[test]
331 #[ignore]
332 fn test_move_dir_between_dirs() -> Result<(), Box<dyn std::error::Error>> {
333     let root_dir = get_test_root_dir()?;
334     let dir_branch_num = uuid::Uuid::now_v7().to_string();
335     let from_dir_path = root_dir.join(format!("test_10_from_dir_{}", dir_branch_num));
336     let to_dir_path = root_dir.join(format!("test_10_to_dir_{}", dir_branch_num));
337     let sub_dir_path = from_dir_path.join(format!("test_10_subdir_{}", dir_branch_num));
338     let test_file_path = sub_dir_path.join(format!("test_10_file_{}", dir_branch_num));
339     let moved_dir_path = to_dir_path.join(format!("test_10_subdir_{}", dir_branch_num));
340     let moved_file_path = moved_dir_path.join(format!("test_10_file_{}", dir_branch_num));
341 
342     fs::create_dir(&from_dir_path)?;
343     fs::create_dir(&to_dir_path)?;
344     fs::create_dir(&sub_dir_path)?;
345     {
346         let mut file = fs::File::create(&test_file_path)?;
347         file.write_all(dir_branch_num.as_bytes())?;
348     }
349 
350     fs::rename(&sub_dir_path, &moved_dir_path)?;
351     assert!(!sub_dir_path.try_exists()?, "元のディレクトリが存在する。");
352     assert!(
353         moved_dir_path.try_exists()?,
354         "移動後のディレクトリが存在しない。"
355     );
356     assert!(!test_file_path.try_exists()?, "元のファイルが存在する。");
357     assert!(
358         moved_file_path.try_exists()?,
359         "移動後のファイルが存在しない。"
360     );
361     {
```

```
362         let content = fs::read_to_string(&moved_file_path)?;
363         assert_eq!(
364             content, dir_branch_num,
365             "書き込んだ内容と読み出した内容が一致しない。"
366         );
367     }
368 
369     fs::remove_dir(&from_dir_path)?;
370     fs::remove_dir_all(&to_dir_path)?;
371     Ok(())
372 }
373 
374 /// [11] ディレクトリから既存ファイル名への rename とその逆の挙動
375 #[test]
376 #[ignore]
377 fn test_rename_dir_to_existing_file() -> Result<(), Box<dyn std::error::Error>> {
378     let root_dir = get_test_root_dir()?;
379     let branch_num = uuid::Uuid::now_v7().to_string();
380     let dir_path = root_dir.join(format!("test_11_dir_{}", branch_num));
381     let file_path = root_dir.join(format!("test_11_file_{}", branch_num));
382 
383     fs::create_dir(&dir_path)?;
384     {
385         let mut file = fs::File::create(&file_path)?;
386         file.write_all(branch_num.as_bytes())?;
387     }
388 
389     let rename_err = fs::rename(&dir_path, &file_path)
390         .expect_err("ディレクトリから既存ファイル名への rename は失敗するはず。");
391     assert_eq!(
392         rename_err.kind(),
393         std::io::ErrorKind::NotADirectory,
394         "エラー内容が想定外 [{}]",
395         rename_err
396     );
397 
398     let rename_err = fs::rename(&file_path, &dir_path)
399         .expect_err("既存ファイル名からディレクトリへの rename は失敗するはず。");
400     assert_eq!(
401         rename_err.kind(),
402         std::io::ErrorKind::IsADirectory,
403         "エラー内容が想定外 [{}]",
404         rename_err
405     );
406 
407     fs::remove_dir(&dir_path)?;
```

```rust
    fs::remove_file(&file_path)?;

    Ok(())
}

/// [12] ある程度の大きさのファイルを複数生成し、内容を確認する。
#[test]
#[ignore]
fn test_create_and_read_large_files() -> Result<(), Box<dyn std::error::Error>> {
    const FILE_SIZE: usize = 1024 * 1024;
    const FILE_COUNT: usize = 100;

    let root_dir = get_test_root_dir()?;
    let branch_num = uuid::Uuid::now_v7().to_string();
    let dir_path = root_dir.join(format!("test_12_dir_{}", branch_num));
    fs::create_dir(&dir_path)?;

    let mut file_paths = Vec::new();
    let mut hash = Vec::new();
    for i in 0..FILE_COUNT {
        let file_path = dir_path.join(format!("test_12_file_{}_{}", i, branch_num));
        file_paths.push(file_path.clone());
        let mut contents = vec![0u8; FILE_SIZE];
        rand::fill(&mut contents[..]);
        hash.push(blake3::hash(&contents));
        let mut file = fs::File::create_new(file_path)?;
        file.write_all(&contents)?;
    }
    for (path, hash) in file_paths.iter().zip(hash.iter()) {
        let contents = fs::read(path)?;
        let read_hash = blake3::hash(&contents);
        assert_eq!(*hash, read_hash, "ファイルの内容が一致しない: {:?}", path);
    }

    fs::remove_dir_all(&dir_path)?;
    Ok(())
}

/// [13] マルチスレッドでのファイル操作
/// (同一ディレクトリ内で操作はするが、同一ファイルを複数スレッドで同時に操作することはしない)
#[test]
#[ignore]
fn test_multithreaded_file_operations() -> Result<(), Box<dyn std::error::Error>> {
    const THREAD_COUNT: usize = 10;
    let root_dir = get_test_root_dir()?;
    let branch_num = uuid::Uuid::now_v7().to_string();
```

```rust
    let base_dir = root_dir.join(format!("test_13_dir_{}", branch_num));
    fs::create_dir(&base_dir)?;
    let mut join_handles = Vec::new();
    let barrier = Arc::new(Barrier::new(THREAD_COUNT));
    for _i in 0..THREAD_COUNT {
        let join = std::thread::spawn({
            let path = base_dir.clone();
            let barrier = barrier.clone();
            || thread_func(path, barrier)
        });
        join_handles.push(join);
    }
    for join in join_handles {
        match join.join() {
            Ok(result) => result?,
            Err(payload) => std::panic::resume_unwind(payload),
        }
    }
    fs::remove_dir_all(base_dir)?;
    Ok(())
}

fn thread_func(path: impl AsRef<Path>, barrier: Arc<Barrier>) -> std::io::Result<()> {
    barrier.wait();
    let base_path = path.as_ref();
    let branch_num = uuid::Uuid::now_v7().to_string();

    let test_file_path = base_path.join(format!("test_13_file_{}", branch_num));
    {
        let mut file = fs::File::create(&test_file_path)?;
        file.write_all(branch_num.as_bytes())?;
    }
    {
        let content = fs::read_to_string(&test_file_path)?;
        assert_eq!(
            content, branch_num,
            "書き込んだ内容と読み出した内容が一致しない。"
        );
    }
    let test_dir_path = base_path.join(format!("test_13_dir_{}", branch_num));
    let moved_file_path_suffix = test_file_path.strip_prefix(&base_path).unwrap();
    let moved_file_path = test_dir_path.join(moved_file_path_suffix);
    fs::create_dir(&test_dir_path)?;
    fs::rename(&test_file_path, &moved_file_path)?;
    {
        let content = fs::read_to_string(&moved_file_path)?;
```

```
500             assert_eq!(
501                 content, branch_num,
502                 "書き込んだ内容と読み出した内容が一致しない。"
503             );
504         }
505         Ok(())
506     }
```

6.2 共通ユーティリティー tests/common/mod.rs

```rust
use std::path::PathBuf;

/// 環境変数 $SSH_TEST_MOUNT の取得。
/// ここが、テスト時にファイル操作を行うルートディレクトリとなる。
///
///  # Errors
/// 環境変数がない場合、テスト実施不可として、Err を返す。
/// このテストは、cargo make test で実行さる前提である。
/// $SSH_TEST_MOUNT は、Makefile.toml で定義されている。
pub fn get_test_root_dir() -> Result<PathBuf, EnvNotFound> {
    let env = std::env::var("SSH_TEST_MOUNT");
    match env {
        Ok(path) => Ok(PathBuf::from(path)),
        Err(_) => Err(EnvNotFound),
    }
}

#[derive(thiserror::Error, Debug)]
#[error("Not found environment variable.This test can only be run using \"cargo make test\".")]
pub struct EnvNotFound;
```

6.3 テスト用サーバー設定のコンフィグファイルサンプル tests/test.env.sample

これは、cargo make test で使用する環境変数です。sshmount の接続設定を記述します。実際に使用するときは、リモートパスとマウントポイントを記述し、'test.env' にリネームします。

```
SSH_TEST_REMOTE = remote_path:
SSH_TEST_MOUNT = mount_point_path
```

6.4 主にテスト用に設定した cargo-make の make ファイル /Makefile.toml

```
env_files = ["./tests/test.env"]

[tasks.envprint]
script = [
    "echo remote = $SSH_TEST_REMOTE",
    "echo mount = $SSH_TEST_MOUNT",
]

[tasks.build]
dependencies = ["clippy"]

[tasks.pre-sshmount-test]
command = "cargo"
args = ["run", "--", "-d", "--debug", "${SSH_TEST_REMOTE}", "${SSH_TEST_MOUNT}"]
dependencies = ["build"]

[tasks.exec-sshmount-test]
env = { RUST_BACKTRACE = "0" }
ignore_errors = true
command = "cargo"
args = ["test", "--", "--include-ignored",]
dependencies = ["pre-sshmount-test"]

[tasks.post-sshmount-test]
script = [
    '''
    #!/bin/bash
    echo end sshmount_test. unmount ${SSH_TEST_MOUNT}
    sleep 2s
    umount ${SSH_TEST_MOUNT}
    '''
]
dependencies = ["exec-sshmount-test"]

[tasks.test]
clear = true
run_task = "post-sshmount-test"
```